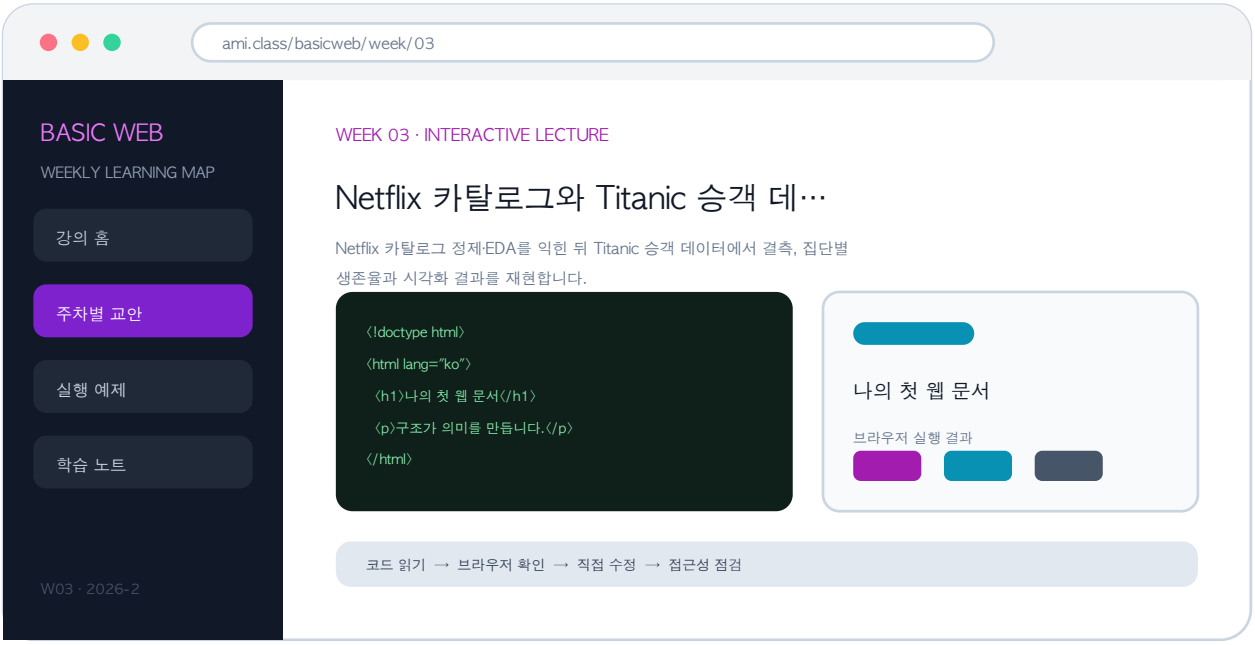
WEEK 03 · 3주차

Netflix 카탈로그와 Titanic 승객 데이터 EDA

AI기반 빅데이터 프로젝트 · 2026-2



승인된 수강생의 학습을 위한 자료입니다. 퀴즈 정답과 해설은 교수자의 별도 공개 전까지 포함하지 않습니다.

3장. Kaggle 공개데이터 수집부터 Netflix 카탈로그 전처리·EDA까지

데이터 분석을 시작할 때는 먼저 질문을 정하고 그 질문에 필요한 열이 파일에 있는지 살펴본다. 이번 장에서는 Netflix 콘텐츠 목록을 읽어 결측값과 여러 값이 한 칸에 들어 있는 열을 정리하고, 집계표와 차트로 탐색적 데이터 분석(EDA)을 해 본다.

Netflix Movies and TV Shows 데이터는 Kaggle에 소개되어 있다. 실습에는 같은 결과를 확인하기 쉽도록 TidyTuesday가 보관한 netflix_titles.csv를 사용한다.

> 학습 경로
>
> 핵심은 Netflix 데이터의 관찰 단위, 품질 진단, 정제, 집계표, 시각화와 해석이다. 심화는 결측 메커니즘, 부트스트랩, 희소행렬, 분산 처리 확장이다. 먼저 핵심 코드를 위에서 아래로 실행한 뒤 심화 상자를 읽는다. 수식은 코드가 수행한 변환을 설명할 때만 사용한다.

학습목표

1. Kaggle의 Dataset, Code, Competition, Discussion, API가 수행하는 역할을 구분할 수 있다.
2. 데이터 설명을 읽고 한 행의 의미와 분석에 필요한 열이 있는지 판단할 수 있다.
3. 분석 질문이 보유 변수로 답할 수 있는지 판단하고 관찰 단위·지표·분모를 정의할 수 있다.
4. 브라우저 다운로드, Kaggle API, 고정 미러 수집의 장단점을 비교할 수 있다.
5. 원본을 변경하지 않고 행·열·기본키·결측을 확인하여 분석 가능한 파일인지 판단할 수 있다.
6. 결측·중복·형식 오류·이상값을 진단하고 정제와 전처리를 구분하여 적용할 수 있다.
7. 범주형·수치형·시간·다중값·텍스트 자료에 적절한 EDA 지표와 차트를 선택할 수 있다.
8. 차트에서 직접 관찰한 사실과 데이터가 뒷받침하지 않는 추정을 구분할 수 있다.

3.1 Netflix 카탈로그 데이터 EDA

3.1.1 Kaggle과 공개 데이터 생태계

3.1.1.1 Kaggle은 무엇을 제공하는가

Kaggle은 단순한 파일 다운로드 사이트가 아니다. 데이터 과학 학습과 분석을 위한 여러 기능을 제공한다.

영역	역할	수업에서 확인할 점
Dataset	데이터 파일과 설명을 제공	한 행과 각 열이 무엇을 나타내는지
Code 또는 Notebook	다른 사람이 작성한 분석 코드를 실행	내가 가진 파일에도 적용되는지
Competition	같은 문제를 여러 방법으로 해결	무엇을 예측하고 어떻게 평가하는지
Discussion	질문과 분석 아이디어를 공유	다른 사람이 발견한 주의점은 무엇인지

영역	역할	수업에서 확인할 점
API	프로그램에서 데이터를 내려받음	수동 다운로드와 무엇이 다른지

이번 수업에서는 먼저 Dataset의 설명을 읽고 파일을 직접 탐색한다. 다른 사람의 Notebook은 아이디어를 얻는 데 유용하지만, 코드 결과를 그대로 답으로 받아들이지 않고 내가 받은 데이터에서 다시 확인한다.

3.1.1.2 분석 목적에 맞는 데이터 고르기

Kaggle에서 Netflix를 검색하면 이름이 비슷한 데이터가 여러 개 나온다. 먼저 한 행이 무엇을 나타내는지와 어떤 열이 있는지를 살펴보자. 이번 실습의 netflix_titles.csv는 영화 또는 TV 프로그램 한 편을 한 행으로 기록한다. 작품 유형(type), 공개 연도(release_year), 등록일(date_added), 장르(listed_in) 등 12개 열과 7,787개 행이 있다.

이 파일로는 “영화와 TV 프로그램 중 어느 쪽이 더 많이 등록되어 있는가?”를 분석할 수 있다. 반면 “어느 작품이 가장 많이 시청되었는가?”에는 답할 수 없다. 시청 횟수나 사용자 기록이 없기 때문이다. 데이터를 고를 때는 파일의 크기나 인기도보다 내 질문에 필요한 열이 있는지를 먼저 확인한다. 다음 절에서는 실제 행을 보면서 각 열이 뜻하는 바를 살펴본다.

실습 파일과 설명은 Netflix 데이터 설명에서 확인할 수 있다. Kaggle의 원 데이터셋 페이지에는 다른 시점의 파일이 표시될 수 있으므로, 이 장의 코드에는 안내된 netflix_titles.csv를 사용한다.

3.1.1.3 파일을 받기 전 확인할 것

데이터 설명 페이지에서 무엇을 기록한 데이터인지, 필요한 열이 있는지, 파일을 이용할 수 있는 조건이 무엇인지 확인한다. 파일을 받은 뒤에는 df.shape와 df.columns를 출력해 수업 예제의 7,787행·12열 및 열 이름과 비교한다. 다르면 이후 예제의 숫자가 달라질 수 있으므로 파일 확인 방법부터 살펴본다.

3.1.2 Netflix 데이터와 분석 질문

3.1.2.1 관찰 단위와 12개 열

한 행은 Netflix 카탈로그에 등재된 영화 또는 TV 프로그램 한 건의 메타데이터다. 사용자 한 명의 시청 기록이 아니다.

먼저 수업용 CSV의 실제 첫 세 행을 작품별로 읽어 보자. 3%, 7:19, 23:59는 계산 결과가 아니라 파일의 title 셀에 기록된 작품명이다. 이 표의 한 행에 있는 값은 모두 같은 작품에 속한다.

작품 ID	유형	실제 작품명 (title)	제작 국가	공개 연도	시청 등급	길이
s1	TV Show	3%	Brazil	2020	TV-MA	4 Seasons
s2	Movie	7:19	Mexico	2016	TV-MA	93 min
s3	Movie	23:59	Singapore	2011	R	78 min

예를 들어 s2는 제목이 7:19인 영화이고 상영시간은 93분이다. 제목을 시간이나 비율로 해석하면 안 된다. s1의 감독은 빈칸이며, s2의 감독은 Jorge Michel Grau, s3의 감독은 Gilbert Chan이다. 수업 파일의 나머지 셀도 아래 코드를 실행하면 작품별로 확인할 수 있다.

열	의미와 분석 시 주의점
show_id	콘텐츠 식별자. 결측·중복이 없어야 한다.
type	콘텐츠 유형. Movie 또는 TV Show다.
title	작품명. 동명 작품이 있을 수 있어 제목만으로 중복 삭제하지 않는다.
director	감독. 결측이 있고 한 셀에 여러 명이 들어갈 수 있다.
cast	출연진. 쉼표로 구분된 다중값이다.
country	제작 국가. 공동 제작은 여러 국가가 한 셀에 기록된다.
date_added	Netflix 등록일이며 제작일·시청일이 아니다.
release_year	최초 공개 연도. 등록 연도와 구분한다.
rating	시청 등급이며 사용자 평점이 아니다.
duration	TV는 시즌, 영화는 분 단위이므로 분리 분석한다.
listed_in	쉼표로 구분된 다중 장르다.
description	작품 소개문. 시청 후기나 사용자 반응이 아니다.

```
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

from netflix_pipeline import normalize_multi_values, prepare

RAW = Path("data/raw/netflix_titles.csv")
df = pd.read_csv(RAW, dtype={"show_id": "string"})
clean = prepare(df)
genres = normalize_multi_values(clean, "listed_in")
genre_rating = genres.merge(clean[["show_id", "rating"]], on="show_id", how="left")

print(df.head(3)[["show_id", "type", "title", "country", "release_year", "rating",
"duration"]].to_string(index=False))
print(df.head(3).T.to_string())  # 첫 세 작품의 12개 열을 모두 확인
```

이 첫 셀은 이후 모든 예제가 사용하는 df, clean, genres, genre_rating을 만든다. 새 커널에서 이 셀부터 순서대로 실행하면 숨은 변수가 남지 않는다. 파일을 찾지 못하면 Notebook의 현재 폴더를 실습 폴더로 바꾸거나 RAW를 실제 상대경로로 수정한다.

이 데이터는 무작위로 뽑은 Netflix 이용자 표본이 아니라 특정 시점에 수집된 카탈로그 스냅샷 7,787건이다. 따라서 분석 대상은 파일에 남아 있는 콘텐츠이고, 관심 모집단을 “모든 시점의 Netflix 콘텐츠”나 “전체 이용자”로 확대할 수 없다.

측정 수준	열 예	가능한 요약	주의점
명목형	type, rating, 장르	빈도, 구성비, 교차표	범주 사이에 크기 순서를 임의로 부여하지 않는다.
이산형 수치	TV Show의 시즌 수	중앙값, 빈도표	영화의 분 단위와 합치지 않는다.

측정 수준	열 예	가능한 요약	주의점
연속형에 가까운 수치	Movie 상영시간(분)	평균, 중앙값, 표준편차, 사분위수	긴 꼬리와 극단 관측을 함께 본다.
시간	date_added, release_year	연·월별 건수, 시차	등록일과 공개일은 서로 다른 사건이다.

3.1.2.2 답할 수 있는 질문과 답할 수 없는 질문

이 데이터로 답할 수 있는 질문은 카탈로그 구성과 메타데이터 품질에 관한 것이다.

- 데이터 스냅샷에서 Movie와 TV Show는 각각 몇 건인가?
- date_added가 기록된 콘텐츠는 연도·월별로 몇 건인가?
- 영화 상영시간과 TV Show 시즌 수의 분포는 어떠한가?
- 정규화한 장르·국가·감독·배우는 몇 개 콘텐츠와 연결되는가?
- 장르와 등급 조합은 데이터 안에서 어떻게 분포하는가?
- 어떤 열의 결측률이 높고 콘텐츠 유형별로 차이가 있는가?

다음 질문에는 시청·사용자·비용 데이터가 추가로 필요하다.

- 가장 인기 있거나 선호되는 장르는 무엇인가?
- 어떤 콘텐츠가 가장 많이 시청되었는가?
- 콘텐츠 등록 증가가 가입자 증가를 일으켰는가?
- 특정 작품이 투자 대비 성공했는가?
- 어떤 콘텐츠를 추천해야 하는가?

〔해답 설계〕 인기 장르와 최다 시청 콘텐츠

작품별 재생 로그에 user_id, show_id, played_at, watch_minutes, completed가 있어야 한다. 인기 장르는 고유 시청자 수·총 시청 시간·완주율을 함께 비교하고, 최다 시청 콘텐츠는 같은 사용자의 반복 재생을 어떻게 셀지 먼저 정한다. 현재 카탈로그의 장르별 작품 수는 “공급량”일 뿐 “인기”가 아니다.

〔해답 설계〕 콘텐츠 등록과 가입자 증가

날짜별 신규 가입·해지·활성 사용자, 콘텐츠 등록일, 프로모션과 가격 변경 자료를 결합한다. 단순 상관관계로 원인을 단정하지 않고 시계열의 선후 관계, 계절성, 캠페인 영향을 통제하거나 A/B 실험·준실험으로 인과효과를 추정한다.

〔해답 설계〕 작품의 투자 대비 성공

제작비·라이선스비·마케팅비와 작품별 신규 가입, 유지, 광고·구독 매출 기여가 필요하다. 기여이익 ÷ 총투자비처럼 성공 지표와 귀속 기간을 먼저 합의해야 하며, 카탈로그 메타데이터만으로 ROI를 계산할 수 없다.

〔해답 설계〕 추천할 콘텐츠

사용자-콘텐츠 시청·검색·좋아요·완주 기록을 시간순으로 학습/검증 데이터로 나눈다. 인기 기준선과 개인화 모델을 Recall@K, NDCG@K, 다양성으로 비교하고, 신규 사용자·신규 작품 문제와 개인정보 보호를 함께 점검한다.

〔AI 함정〕 rating은 별점이 아니라 시청 등급이다. 장르 레코드가 많다는 사실은 그 장르가 인기 있다는 뜻이 아니다.

3.1.2.3 질문-지표-차트-결론 카드

EDA를 시작하기 전에 질문을 측정 가능한 형태로 바꾼다.

질문	관찰 단위	지표·분모	차트	허용되는 결론
유형별 구성은?	콘텐츠	유형별 레코드 수 / 전체 유효 레코드	정렬 막대	스냅샷의 구성
등록 추이는?	콘텐츠	연도별 date_added 유효 건수	선 또는 막대	기록된 등록 건수 변화
장르와 등급 조합은?	콘텐츠-장르	조합 건수와 장르 내 구성비	히트맵·100% 누적막대	데이터 내 구성 차이
영화 길이 분포는?	Movie 콘텐츠	유효한 분 단위 duration	히스토그램·박스플롯	중앙값·산포·극단 관측

차트를 먼저 고른 뒤 의미를 붙이지 않는다. 질문, 관찰 단위, 분석 유효 N, 분자와 분모를 먼저 정한다.

장르처럼 한 작품이 여러 범주를 갖는 열에는 특히 두 분모가 있다. 특정 장르와 연결된 고유 작품 수 ÷ 전체 작품 수는 그 장르의 작품 보유율이고, 해당 장르 관계 수 ÷ 전체 장르 관계 수는 관계 구성비다. 복수 장르 때문에 장르별 작품 보유율의 합은 100%를 넘을 수 있다. 둘은 질문이 다르므로 표와 차트에 어떤 분모를 썼는지 표시한다.

3.1.3 재현 가능한 수집과 원본 보존

3.1.3.1 세 가지 수집 방식

방식	장점	위험·한계
브라우저 다운로드	초심자가 파일과 설명을 함께 확인하기 쉬움	클릭 절차와 버전 기록이 빠지기 쉬움
Kaggle API	owner/slug로 자동화 가능	API 토큰 보호와 버전 식별 필요
고정 미러 URL	수업에서 접근·재실행이 쉬움	원출처·라이선스·동기화 시점을 별도 기록해야 함

Kaggle API 인증 파일이나 토큰은 Git 저장소, Notebook 출력, 제출물에 넣지 않는다. 내려받은 뒤에는 우선 열 이름, 행·열 수, 기본키 중복과 결측을 확인한다. 체크섬은 모든 분석 단계에서 반복할 내용이 아니라, 교수자와

학생이 동일한 수업용 스냅샷을 받았는지 최초 1회 확인할 때만 사용한다.

이번 실습의 계보는 다음과 같다.

```
Kaggle Netflix Movies and TV Shows
  └── TidyTuesday 2021-04-20 미러
      └── 수업용 고정 스냅샷: netflix_titles.csv
```

3.1.3.2 Conda 환경은 실습 준비 수단이다

환경 구축은 독립적인 이론 장이 아니라 같은 코드를 재실행하기 위한 준비 단계다.

```
conda env create -f environment.yml
conda activate bdp-netflix-week03
python --version
python -c "import pandas, matplotlib, seaborn; print(pandas.__version__)"
```

base 환경에 임의 설치하거나 Notebook 셀마다 pip install을 반복하지 않는다. 패키지 버전이 바뀌면 날짜 파싱, 범주 정렬, 차트 스타일과 테스트 결과가 달라질 수 있다.

3.1.3.3 폴더 규약과 원본 불변성

```
data/inbox/      # 다운로드 직후 임시 파일
data/raw/        # 검증을 통과한 불변 원본
data/processed/  # 정제·전처리된 파생 데이터
output/          # 프로파일, 품질 보고서, 차트
```

ZIP을 받았다면 ZIP과 추출 CSV 중 무엇을 원본으로 관리할지 선언하고 필요하면 둘 다 해시를 기록한다. raw 파일은 수정하거나 덮어쓰지 않는다. 새로운 스냅샷은 새 버전 디렉터리에 등록한다.

3.1.3.4 수업 파일을 여는 첫 확인

복잡한 매니페스트를 작성하기보다 분석을 망가뜨릴 가능성이 큰 항목부터 확인한다.

항목	기대값
행·열	7,787행 x 12열
기본키 후보	show_id
콘텐츠 유형	Movie, TV Show
날짜	date_added
기간	duration

열 이름·행 수가 달라지면 이후 코드가 다른 결과를 만들 수 있으므로 먼저 원인을 확인한다. 수업용 고정 파일의 체크섬은 7a3aaf…fc143이며, 다른 값이면 파일이 새 버전이거나 전송 중 변경됐는지 교수자에게 확인한다. 401·403·404 응답이나 HTML 오류 페이지를 CSV로 저장한 경우는 정상 데이터가 아니다.

```
python netflix_pipeline.py --download
# 또는
python netflix_pipeline.py --source-file /path/to/netflix_titles.csv
```

3.1.4 데이터를 만지기 전 품질 프로파일링

프로파일링은 처리 방법을 정하기 위한 진단이다. 먼저 원본 상태를 측정하고, 그 뒤에 처리 규칙을 결정한다.

3.1.4.1 구조·키·중복

```
print(df.head())
print(df.sample(5, random_state=42))
print(df.shape)
df.info()
print(df["show_id"].isna().sum())
print(df["show_id"].duplicated().sum())
print(df.duplicated().sum())
print(df.duplicated(subset=["title", "release_year"], keep=False).sum())
```

- show_id 결측·중복은 키 계약 위반이다.
- 전체 행 중복은 동일 레코드의 중복 적재 후보다.
- 제목 중복은 재개봉, 동명 작품 또는 다른 유형일 수 있으므로 자동 삭제하지 않는다.

3.1.4.2 결측·빈 문자열·범주

```
blank_count = df.select_dtypes(include=["object", "string"]).apply(
    lambda column: column.astype("string").str.strip().eq("").sum()
)
profile = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "null_count": df.isna().sum(),
    "null_rate": df.isna().mean(),
    "blank_count": blank_count.reindex(df.columns, fill_value=0),
    "nunique": df.nunique(dropna=True),
})
print(profile.sort_values("null_rate", ascending=False))
print(df["type"].value_counts(dropna=False))
print(df["rating"].value_counts(dropna=False))

plot_profile = profile.sort_values("null_rate")
ax = plot_profile["null_rate"].plot.barh(figsize=(8, 5), color="#2878b5")
ax.set(title="열별 결측률", xlabel="결측 비율", ylabel="열")
plt.tight_layout()
plt.show()
```

실행 결과에서 director 2,389건, cast 718건, country 507건이 결측이다. 반면 식별자·유형·제목·공개연도·기간·장르·설명은 결측이 없다. 따라서 감독 열을 통째로 삭제하기보다 감독이 필요한 분석에서만 분모인 분석 유효 N을 별도로 밝힌다.

NaN만 결측 후보가 아니다. 빈 문자열과 공백 문자열도 별도로 측정한다. 결측률이 높은 열을 무조건 삭제하면 감독·출연진 정보가 덜 기록된 시기나 유형이 선택적으로 제외될 수 있다.

통계적으로는 결측 원인을 세 수준으로 생각할 수 있다. 다른 관측값과 무관하게 빠진 경우를 MCAR, 관측된 유형·연도 등에 따라 빠질 가능성이 달라지는 경우를 MAR, 빠진 값 자체와 관련되어 누락되는 경우를 MNAR이라고 한다. 이 파일만 보고 셋 중 하나를 확정할 수는 없다. 다만 아래처럼 유형별 결측률을 비교하면 “모든 행에서 비슷하게 빠졌는가”라는 첫 진단을 할 수 있다. 유형별 차이는 MCAR을 의심할 단서지만, 차이가 없다고 MCAR이 증명되는 것도 아니다. 삭제 전후의 유형 구성과 분석 유효 N을 반드시 비교한다.

3.1.4.3 날짜·기간·도메인 규칙

다음 규칙은 진단 대상이다.

- type은 Movie 또는 TV Show인가?
- release_year는 합리적인 범위인가?

- 비어 있지 않은 date_added가 지정 형식으로 파싱되는가?
- Movie의 duration은 분 단위인가?
- TV Show의 duration은 시즌 단위인가?
- release_year 〉 added_year인 행은 실제 오류인가, 데이터 의미 차이인가?

통계적 극단값은 자동 삭제 대상이 아니다. 300분 영화는 잘못 입력된 값일 수도 있고 실제 긴 작품일 수도 있다. 원문과 외부 근거를 확인한 뒤 오류·희귀 정상값·분석상 영향값을 구분한다.

3.1.4.4 중단 조건과 경고 조건

검사	등급	처리
수업 고정본의 체크섬 불일치	최초 수집 시 중단	교수자가 지정한 파일과 같은 버전인지 확인
열 누락·순서 불일치	중단	스키마 변경 검토
show_id 결측·중복	중단	키 정의와 원본 확인
날짜·기간 형식 오류	경고 또는 중단	원문과 실패 건수 기록
일반 결측	경고	분석별 처리와 유효 N 기록
극단값	검토	자동 삭제 금지

3.1.5 데이터 정제

3.1.5.1 정제·전처리·EDA의 차이

구분	목적	Netflix 예시
정제	오류·불일치·표기 문제를 품질 규칙에 맞춤	공백 제거, 빈 문자열 통일, 형식 오류 표시
전처리	분석 가능한 구조와 특징으로 변환	날짜 파생, duration 분리, 다중값 explode
EDA	분포·패턴·관계를 탐색하고 질문을 갱신	유형·연도·장르·기간 분포

정제는 “결측을 모두 없애는 작업”이 아니다. 문제를 측정하고, 처리 선택의 근거와 데이터 손실을 기록하는 과정이다.

3.1.5.2 문자열과 결측 처리

문자열 앞뒤 공백을 제거하되 원본 파일은 보존한다. 결측 처리에는 네 단계를 적용한다.

1. 결측의 개수·비율·위치를 측정한다.
2. 결측이 의미하는 바와 발생 원인을 추정한다.
3. 분석 질문에 따라 유지·제외·대체·별도 범주화를 선택한다.
4. 처리 후 유효 표본 수와 분포 변화를 기록한다.

Unknown을 넣는 것은 결측을 해결한 것이 아니라 결측을 명시적 범주로 부호화한 것이다. 감독 순위에 Unknown을 포함하면 실제 감독처럼 보일 수 있으므로 집계에서 분리한다.

3.1.5.3 중복과 이상값

전체 행 중복과 키 중복은 오류 후보이지만 제목 중복은 자동 삭제하지 않는다. 삭제가 필요하면 기준, 대상 식별자, 삭제 건수, 처리 전후 행 수를 변환 로그에 기록한다.

이상값 역시 박스플롯 밖에 있다는 이유로 제거하지 않는다. 도메인 규칙 위반인지, 희귀하지만 정상인지, 분석 결과에 큰 영향을 주는 관측인지 구분한다.

3.1.6 분석 목적 전처리

3.1.6.1 날짜 파생

원문 date_added를 보존하고 파생 열을 만든다. 원문이 있는데 파싱되지 않은 행은 실패로 집계한다.

```
clean["date_added_parsed"] = pd.to_datetime(
    clean["date_added"], format="%B %d, %Y", errors="coerce"
)
clean["added_year"] = clean["date_added_parsed"].dt.year
clean["added_month"] = clean["date_added_parsed"].dt.month
```

3.1.6.2 duration의 값과 단위 분리

영화는 분, TV Show는 시즌 수다. 숫자만 추출해 한 평균으로 합치면 의미가 없다.

```
parts = clean["duration"].str.extract(r"^(\d+)\s+(min|Season|Seasons)$")
clean["duration_value"] = pd.to_numeric(parts[0], errors="coerce")
clean["duration_unit"] = parts[1]
```

영화는 duration_unit == "min", TV Show는 Season 또는 Seasons인지 각각 검사한다.

3.1.6.3 다중값 열의 장형 정규화

director, cast, country, listed_in은 하나의 셀에 복수 값이 들어 있다. 문자열 전체를 하나의 범주로 세지 않고 장형 관계 테이블로 만든다.

```
genres = (
    clean[["show_id", "listed_in"]]
    .dropna(subset=["listed_in"])
    .assign(value=lambda x: x["listed_in"].str.split(","))
    .explode("value")
    .assign(value=lambda x: x["value"].str.strip())
    .drop_duplicates()
)
```

한 콘텐츠가 여러 장르에 속하므로 장르 관계 수의 합은 콘텐츠 행 수보다 클 수 있다. 이는 오류가 아니라 다대다 관계의 결과다. 다만 쉼표가 사람 이름 자체에 포함되거나 표기가 다른 문제는 고급 개체명 정합 과제로 남는다.

선형대수 관점에서는 이를 콘텐츠×장르 소속행렬로 볼 수 있다. 행은 콘텐츠 7,787건, 열은 고유 장르이고, 어떤 콘텐츠가 어떤 장르에 속하면 1, 아니면 0이다. 한 행에 1이 여러 개 있을 수 있으며 대부분의 칸은 0이므로 희소행렬이다. 행 합은 작품 하나의 장르 수, 열 합은 장르별 작품 수가 된다. 현재 실습은 행렬을 직접 만들지 않고 장형 표로 같은 정보를 보존하지만, 추천·유사도 계산으로 확장할 때 이 표현이 연결된다.

실제 고정본을 펼치면 감독 관계 6,114건(5,398작품), 출연진 관계 55,953건(7,069작품), 국가 관계 9,063건(7,280작품), 장르 관계 17,071건(7,787작품)이 된다. 관계 수가 작품 수보다 큰 것은 다중값 열을 펼친 결과다.

3.1.6.4 텍스트 전처리의 최소 원칙

description은 먼저 결측률, 문자열 길이, 언어와 반복 문구를 살핀다. 소문자화·구두점 제거·불용어 제거는 분석 목적을 적은 뒤 적용한다. 워드클라우드보다 단어 빈도표를 먼저 만들고, 자주 나온 단어를 곧바로 핵심 주제나 중요도로 해석하지 않는다.

3.1.7 단변량 EDA

3.1.7.1 범주형 변수

범주형 변수는 빈도와 구성비를 함께 본다. 범주가 많을 때 파이차트보다 정렬된 막대그래프가 비교에 유리하다.

```
type_table = (
    clean["type"].value_counts(dropna=False)
    .rename_axis("type").reset_index(name="records")
)
type_table["share"] = type_table["records"] / type_table["records"].sum()

ax = sns.barplot(data=type_table, x="type", y="records", hue="type", legend=False)
ax.set(title="콘텐츠 유형별 카탈로그 건수", xlabel="콘텐츠 유형", ylabel="건수")
for container in ax.containers:
    ax.bar_label(container, fmt="%d")
plt.tight_layout()
plt.show()
```

실제 결과는 Movie 5,377건(69.1%), TV Show 2,410건(30.9%)이다. 이 비율은 시청 선호가 아니라 이 스냅샷에 남아 있는 카탈로그의 구성비다.

type, rating, 정규화한 listed_in을 분석할 때 유효 N과 결측 제외 N을 함께 표기한다. 상위 N만 보이면 나머지 범주의 합도 표시한다.

3.1.7.2 수치형 변수

평균만으로 분포를 요약하지 않는다. 중앙값, 사분위수, 범위, 표준편차와 분포 모양을 함께 본다.

- 영화 상영시간: 히스토그램과 박스플롯
- TV Show 시즌 수: 이산형 빈도 막대
- 제작 연도: 히스토그램과 관측 범위

Movie와 TV Show의 duration은 단위가 다르므로 별도 표와 차트를 만든다. 박스플롯의 점은 검토 후보이지 자동 오류가 아니다.

```
movies = clean.loc[
    clean["type"].eq("Movie") & clean["duration_unit"].eq("min"),
    ["title", "duration_value"],
].dropna()
from netflix_pipeline import bootstrap_median_interval, duration_statistics

print(duration_statistics(clean).round(2))
median_low, median_high = bootstrap_median_interval(
    movies["duration_value"], repetitions=3000, seed=42
)
print(f"관측 스냅샷 재표집 중앙값 범위: {median_low:.1f}~{median_high:.1f}분")

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.histplot(data=movies, x="duration_value", bins=30, ax=axes[0], color="#2878b5")
sns.boxplot(data=movies, x="duration_value", ax=axes[1], color="#f2b134")
axes[0].set(title="영화 상영시간 분포", xlabel="분")
```

```
axes[1].set(title="영화 상영시간 상자그림", xlabel="분")
plt.tight_layout()
plt.show()
```

5,377편의 영화 상영시간 중앙값은 98분, 평균은 99.3분이며 1·3사분위수는 86분과 114분이다. 최솟값 3분과 최댓값 312분은 차트에서 확인할 검토 대상이지 자동 삭제 대상이 아니다.

평균은 모든 값을 사용하므로 긴 영화의 영향을 받고, 중앙값은 정렬했을 때 가운데 값이라 긴 꼬리에 덜 민감하다. 표준편차는 값들이 평균에서 얼마나 퍼져 있는지를 원래 단위인 분으로 요약하고, 사분위범위는 가운데 50%의 폭을 나타낸다. 이 파일의 5,377편을 분석 대상 전체로 보면 모집단 표준편차(ddof=0)를, 더 큰 모집단에서 추출된 표본으로 가정하면 표본 표준편차(ddof=1)를 쓸 수 있다. 본 장은 스냅샷 자체를 기술하므로 전자를 기본으로 보고한다.

부트스트랩 범위는 현재 관측값을 복원추출했을 때 중앙값이 얼마나 흔들리는지 보여주는 민감도 점검이다. 무작위 표본설계가 없는 카탈로그 스냅샷이므로 Netflix 전체 콘텐츠 모집단의 신뢰구간이라고 부르지 않는다.

3.1.7.3 결측 자체의 EDA

결측률 막대그래프와 콘텐츠 유형별 결측률을 비교한다. 특정 열의 결측이 특정 유형이나 연도에 집중된다면 완전 무작위 결측으로 가정할 수 없다. 결측을 삭제한 분석은 처리 전후 집단 구성과 유효 N을 보고한다.

```
missing_by_type = clean.groupby("type")[["director", "cast", "country"]].apply(
    lambda part: part.isna().mean()
)
print(missing_by_type)
missing_by_type.T.plot.bar(figsize=(9, 4), color=["#2878b5", "#f2b134"])
plt.title("콘텐츠 유형별 주요 열 결측률")
plt.ylabel("결측 비율")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

3.1.8 이변량·다변량 EDA

3.1.8.1 범주형×범주형

type × rating, genre × rating은 교차표로 시작한다. 아래 장르 표의 관찰 단위는 콘텐츠-장르 관계다. 한 작품이 둘 이상의 장르에 나타날 수 있으므로 이 표의 관계 수를 독립된 콘텐츠 수나 시청자 수로 읽지 않는다. 결측 등급을 Missing으로 별도 범주화해 교차표에서 조용히 제외되지 않게 한다.

```
rating_for_table = genre_rating["rating"].fillna("Missing")
counts = pd.crosstab(genre_rating["value"], rating_for_table)
row_share = counts.div(counts.sum(axis=1), axis=0)
print(f"장르 관계 전체 N={len(genre_rating):,}; 등급 미기록 관계
N={genre_rating['rating'].isna().sum():,}")

top = counts.loc[counts.sum(axis=1).nlargest(12).index]
top_share = row_share.loc[top.index]
print(top.assign(total=top.sum(axis=1)))
print(top_share.round(3))
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
sns.heatmap(top, cmap="Blues", linewidths=.4, ax=axes[0])
sns.heatmap(top_share, cmap="Purples", linewidths=.4, ax=axes[1])
axes[0].set(title="상위 장르×등급 건수", xlabel="시청 등급", ylabel="장르")
axes[1].set(title="각 장르 내부 등급 구성비", xlabel="시청 등급", ylabel="장르")
fig.tight_layout()
plt.show()
```

원시 건수는 전체 규모를, 행 비율은 각 장르 내부의 등급 구성을 보여준다. 관측 관계 수가 많은 장르가 히트맵을 지배하지 않도록 질문에 맞는 표를 선택한다. 각 행의 분모는 그 장르의 total이며, 결측을 제외한 유효 N만 보고 싶다면 Missing 열을 제외한 수를 별도로 제시해야 한다. 조합이 많다는 사실을 강한 연관성이나 타깃 전략으로 단정하지 않는다.

건수 히트맵에서 진한 칸은 “해당 조합의 작품이 많다”는 뜻이다. 행 구성비 히트맵에서 진한 칸은 “그 장르 작품 중 해당 등급의 비중이 크다”는 뜻이다. 두 표가 다르게 보이는 이유는 분모가 다르기 때문이다. 장르와 등급의 통계적 연관성을 수치화하려면 카이제곱 검정과 Cramr's V를 추가할 수 있지만, 범주 통합·기대빈도 조건·스냅샷의 비확률성을 먼저 검토해야 하므로 본 장에서는 심화로 남긴다.

3.1.8.2 범주형×수치형

등급별 영화 상영시간을 박스플롯이나 바이올린플롯으로 비교한다. 각 그룹의 표본 수를 함께 제시하고, 평균선만 비교하지 않는다. 그룹 크기가 매우 작으면 차이를 일반화하지 않는다.

비교 전에 Movie만 선택하고 분 단위 파싱에 성공한 행만 남겼다는 조건을 기록한다. 등급별 중앙값과 사분위범위가 겹치는지, 긴 꼬리와 극단 관측이 어느 집단에 집중되는지를 살핀다. 차이가 보이더라도 등급이 상영시간을 결정한다고 표현하지 않는다. 제작 시기, 장르, 국가처럼 함께 변하는 요인을 통제하지 않은 기술적 비교이기 때문이다.

```
movie_rating = clean.loc[
    clean["type"].eq("Movie") & clean["duration_unit"].eq("min"),
    ["rating", "duration_value"],
].dropna()
order = movie_rating["rating"].value_counts().head(10).index
group_summary = (
    movie_rating[movie_rating["rating"].isin(order)]
    .groupby("rating")["duration_value"]
    .agg(valid_n="count", median="median", mean="mean",
         population_sd=lambda s: s.std(ddof=0))
    .sort_values("valid_n", ascending=False)
)
print(group_summary.round(1))
plt.figure(figsize=(11, 5))
sns.boxplot(data=movie_rating[movie_rating["rating"].isin(order)],
            x="rating", y="duration_value", order=order, color="#8ecae6")
plt.title("주요 시청 등급별 영화 상영시간")
plt.xlabel("시청 등급")
plt.ylabel("분")
plt.xticks(rotation=35)
plt.tight_layout()
plt.show()
```

먼저 valid_n으로 그룹 크기를 확인하고 중앙값과 상자 높이(IQR)를 비교한다. 상자가 겹치거나 표본이 매우 작은 등급은 눈에 보이는 차이를 과장하지 않는다. 이 비교는 등급별 분포를 기술할 뿐, 등급이 상영시간을 변화시킨다는 인과효과를 추정하지 않는다.

3.1.8.3 제작 연도×등록 연도

release_year와 added_year 산점도는 제작 시점과 등록 시점의 차이를 보여준다. 점이 많으면 투명도 또는 hexbin을 사용한다. 다음 파생값이 해석에 더 직접적이다.

```
clean["acquisition_lag"] = clean["added_year"] - clean["release_year"]

lag_summary = clean.groupby("type")["acquisition_lag"].agg(
    valid_n="count", median="median", mean="mean", std="std"
)
```

```
print(lag_summary.round(1))

plt.figure(figsize=(7, 6))
sns.scatterplot(data=clean, x="release_year", y="added_year",
                hue="type", alpha=.25, s=18)
plt.title("공개 연도와 Netflix 등록 연도")
plt.xlabel("최초 공개 연도")
plt.ylabel("Netflix 등록 연도")
plt.tight_layout()
plt.show()
```

두 연도가 가깝다고 Netflix Original인 것은 아니다. Original 여부 열이 없으므로 콘텐츠 조달 전략까지 결론 내릴 수 없다.

두 연도 사이의 Pearson 상관은 직선 관계, Spearman 상관은 순위의 단조 관계를 요약하지만 둘 다 원인을 증명하지 않는다. 또한 acquisition_lag 〈 0인 12건은 계산 오류로 즉시 삭제하지 말고, 공개연도와 등록일의 정의·원문을 재확인해야 하는 품질 후보로 기록한다.

3.1.9 시계열 EDA

3.1.9.1 제작 시점과 등록 시점

release_year는 작품의 최초 공개 연도이고 date_added는 카탈로그 등록일이다. 서로 다른 사건이므로 한 시계열처럼 섞지 않는다.

예를 들어 2010년에 공개된 영화가 2019년에 카탈로그에 등록될 수 있다. 이때 2010은 콘텐츠 제작·공개 시점을, 2019는 플랫폼 등록 시점을 나타낸다. 두 열을 같은 의미의 연도로 집계하면 제작 추세와 등록 추세가 뒤섞인다. 분석표와 차트 제목에는 반드시 어떤 사건의 연도인지 명시한다.

3.1.9.2 연도·월별 등록 패턴

- 연도별 등록 건수: 선 또는 막대그래프
- 월별 등록 구성: 월 순서로 정렬한 막대그래프
- 연도×월: 건수 또는 연도 내 구성비 히트맵

```
annual = (
    clean.dropna(subset=["added_year"])
    .groupby(["added_year", "type"]).size()
    .rename("records").reset_index()
)
sns.lineplot(data=annual, x="added_year", y="records", hue="type", marker="o")
print(annual.to_string(index=False))
cutoff = clean["date_added_parsed"].max()
print(f"관측된 마지막 등록일: {cutoff:%Y-%m-%d}; {cutoff.year}년은 부분 관측")
plt.title(f"현재 스냅샷 잔존 콘텐츠의 등록 연도별 건수 ({cutoff.year}년 부분 관측)")
plt.xlabel("date_added의 연도")
plt.ylabel("건수")
plt.tight_layout()
plt.show()
```

실제 집계는 2017년 1,225건, 2018년 1,685건, 2019년 2,153건, 2020년 2,009건이다. 2021년은 117건뿐이지만 부분 연도이므로 2020년과 같은 조건으로 비교하지 않는다.

부분 연도는 완전 연도와 직접 비교하지 않는다. 2021년 값이 낮다는 이유로 등록 전략이 축소됐다고 결론 내리면 안 된다.

3.1.9.3 스냅샷의 생존 편향

현재 카탈로그 스냅샷에는 과거에 등록되었다가 삭제된 콘텐츠가 빠졌을 수 있다. 따라서 date_added 집계는 완전한 역사적 입출고 로그가 아니다. 등록 건수 변화만으로 가입자 성장, 투자 확대, 글로벌 전략을 설명하지 않는다.

3.1.10 텍스트·다중값 EDA

3.1.10.1 설명문 텍스트

텍스트 분석은 문서 수, 결측률, 길이 분포에서 시작한다. 전체 단어 빈도와 장르별 TF-IDF는 서로 다른 질문에 답한다. 워드클라우드는 면적 비교가 어렵고 빈도 차이를 정확히 읽기 어려우므로 보조 그림으로만 사용한다.

토큰화 전에 대소문자, 구두점, 숫자, 작품명, 인명, 불용어를 어떻게 처리할지 규칙을 기록한다. 빈도표에는 단어와 문서 빈도를 함께 제시하여 한 설명문에서 반복된 단어가 전체 결과를 지배하는지 확인한다. 장르별 비교에서는 문서 수가 다르므로 단순 총빈도뿐 아니라 문서당 비율이나 TF-IDF를 함께 보고, 큰 단어를 곧바로 중요한 주제나 이용자 관심으로 해석하지 않는다.

3.1.10.2 배우·감독·국가·장르

장형 정규화 후 콘텐츠 수와 관계 수를 구분한다. 감독 한 명의 작품 수, 배우 한 명의 참여 수, 국가 한 곳과 연결된 콘텐츠 수를 셀 때 중복 관계를 제거한다. 첫 번째 국가나 첫 번째 장르를 대표값으로 삼으면 공동 제작과 복수 장르 정보가 손실된다.

```
top_genres = genres["value"].value_counts().head(10).sort_values()
print(top_genres.rename("고유 작품 수").to_string())
ax = top_genres.plot.barh(figsize=(9, 5), color="#7b2cbf")
ax.set(title="카탈로그에 가장 많이 등장한 장르 10개", xlabel="콘텐츠-장르 관계 수", ylabel="장르")
plt.tight_layout()
plt.show()
```

가장 많이 등장한 범주는 International Movies 2,437건, Dramas 2,106건, Comedies 1,471건이다. 이는 장르별 보유 작품 수이며 사용자 선호 순위가 아니다.

3.1.11 종합 실습

3.1.11.1 데이터 확보와 파이프라인 실행

```
python netflix_pipeline.py --download
python netflix_pipeline.py
python -m json.tool output/quality_report.json | head -n 100
head -n 13 output/profile.csv
find output/charts -maxdepth 1 -type f -print
```

성공하면 품질 보고서와 프로파일 외에 분석 집계표, 11개 차트, 실행 환경·입출력 해시를 담은 run_manifest.json이 생성된다. 차트만 제출하지 말고 차트에 사용한 집계표를 함께 보존해야 숫자를 재확인할 수 있다.

증상	원인 후보	복구 방법
conda: command not found	Conda 미설치 또는 셸 초기화 누락	Miniconda 설치 후 새 터미널을 열고 conda init 상태를 확인한다.

증상	원인 후보	복구 방법
ModuleNotFoundError	다른 Python 환경 실행	conda activate bdp-netflix-week03 후 which python을 확인한다.
CSV를 찾지 못함	현재 폴더 또는 상대경로 오류	실습 폴더에서 실행하고 data/raw/netflix_titles.csv 존재를 확인한다.
체크섬 불일치	다른 버전·HTML 오류 응답·손상 파일	지정한 원출처에서 다시 받고 파일 크기·첫 행·열 이름을 확인한다.
스키마 계약 실패	열 추가·삭제·이름 변경	새 버전을 덮어쓰지 말고 별도 폴더에서 변경점을 검토한다.
한글 글꼴 경고	실행 환경에 글꼴 없음	영문 축 이름을 쓰거나 설치된 한글 글꼴을 명시한다.

3.1.11.2 필수 분석 카드

다음 다섯 질문 중 최소 세 개를 선택한다.

1. Movie 상영시간의 분포는 어떠한가?
2. 등급별 Movie 상영시간의 분포는 어떻게 다른가?
3. 연도별 카탈로그 등록 건수는 어떻게 기록되어 있는가?
4. 정규화한 장르 상위 15개의 레코드 수는 얼마인가?
5. 콘텐츠 유형별 결측률은 어떻게 다른가?

각 질문은 다음 형식으로 제출한다.

1. 분석 질문
2. 관찰 단위와 분석 대상
3. 사용할 열과 전처리 규칙
4. 지표의 분자·분모와 유효 N
5. 선택한 차트와 선택 이유
6. 차트에서 직접 관찰한 사실
7. 가능한 결론
8. 불가능한 결론
9. 데이터 한계와 추가로 필요한 데이터

3.1.11.3 이 데이터가 커지면 무엇이 달라지는가

현재 CSV는 약 3MB, 7,787행인 교육용 소규모 데이터이므로 pandas 한 프로세스로 충분하다. 빅데이터 분석의 핵심은 작은 파일에 분산 도구를 억지로 쓰는 것이 아니라, 같은 분석 계약이 규모가 커졌을 때 어디서 깨지는지 설명하는 데 있다.

현재 실습	데이터가 메모리를 넘을 때의 확장
CSV 한 파일	Parquet 등 열 지향 형식과 날짜·유형 파티션
pd.read_csv 전체 적재	필요한 열·파티션만 읽거나 청크 처리

현재 실습	데이터가 메모리를 넘을 때의 확장
explode 후 단일 메모리 집계	관계 테이블을 별도 저장하고 분산 explode·groupBy 수행
로컬 정렬·교차표	셔플 비용과 데이터 쏠림을 점검하고 사전 집계
파일 하나의 검사	수집 단계 스키마 계약, 실패 레코드 격리, 관측성 지표

분산 처리에서도 관찰 단위, 키, 분모, 단위, 결측 규칙은 변하지 않는다. 달라지는 것은 저장 형식, 연산 위치, 파티션, 네트워크 셔플과 장애 복구 방식이다.

3.1.11.4 잘못된 해석 교정

다음 표현을 데이터가 직접 보여 주는 문장으로 고친다.

- “가장 대중적인 장르”
- “콘텐츠 제작을 주도하는 국가”
- “하반기 공개 전략”
- “Netflix Original 투자 확대”
- “중심성이 높은 인물은 영향력이 크다”

예: “가장 대중적인 장르” → “이 스냅샷에서 정규화한 장르 관계 레코드가 가장 많은 분류”.

3.1.12 해석 오류 체크리스트

- 빈도 높은 장르를 인기 장르라고 부르지 않았는가?
- date_added를 제작일 또는 시청일로 해석하지 않았는가?
- 카탈로그 레코드 수를 Netflix 전체 보유량으로 확대하지 않았는가?
- 상관관계를 원인으로 표현하지 않았는가?
- 부분 연도와 완전 연도를 직접 비교하지 않았는가?
- 다중값 열을 셀 문자열 그대로 집계하지 않았는가?
- Unknown을 실제 범주처럼 해석하지 않았는가?
- 표본 수가 다른 집단을 비율 없이 건수만으로 비교하지 않았는가?
- 가상 데이터와 실제 데이터를 결합해 실증 결과처럼 서술하지 않았는가?
- 차트 제목·축·단위·유효 N·제외 기준을 적었는가?

3.1.13 제출물과 평가 기준

3.1.13.1 제출물

- 사용한 데이터셋 링크와 받은 날짜, 파일을 선택한 이유를 적은 README.md
- output/quality_report.json
- output/profile.csv
- output/tables/ 의 분모가 명시된 집계표

- output/run_manifest.json
- 정제 데이터와 네 개의 장형 관계 테이블
- EDA 차트와 집계표
- 질문-지표-차트-결론 카드 3개 이상
- week03-observations.md

3.1.13.2 평가 기준

평가 요소	비중
데이터의 관찰 단위와 질문·변수 적합성 설명	20%
품질 진단과 처리 근거	25%
재현 가능한 코드와 산출물	20%
질문에 맞는 지표·차트 선택	20%
사실·추정·한계 구분	15%

3.2 Titanic 승객 데이터 EDA

Titanic 승객 데이터를 직접 읽고 결측을 확인한 뒤 집단별 생존율을 표와 차트로 분석한다. 집계할 때는 각 집단의 인원을 분모로 명시하고, 관찰 결과만으로 생존 원인을 단정하지 않는다.

3.2.1 Kaggle 파일과 실제 행

Kaggle Titanic 대회 데이터에서 train.csv와 test.csv를 내려받는다. 기존 Titanic 노션 자료의 titanic.zip도 같은 수업용 입력으로 사용할 수 있다. 실습 폴더는 practices/week03-titanic/ 이다.

파일	실제 행·열	생존 결과	용도
train.csv	891행 × 12열	Survived 있음	결측·분포·생존율 분석
test.csv	418행 × 11열	Survived 없음	입력 변수 구조 확인

첫 행의 PassengerId=1, Name=Braund, Mr. Owen Harris, Pclass=3, Sex=male, Age=22, Fare=7.25, Survived=0은 같은 승객 한 명의 기록이다. Survived=0은 사망, 1은 생존을 뜻한다. test.csv에는 실제 결과가 없으므로 이 파일만으로 생존율이나 모델 정확도를 계산하지 않는다. gender_submission.csv는 제출 형식 예시이며 정답표가 아니다.

열	의미	분석할 때 구분할 점
PassengerId	승객 식별자	번호의 평균·상관계수는 해석하지 않는다
Pclass	1·2·3등급	숫자로 저장된 순서형 범주다
Age, Fare	나이(세), 승선 요금	결측과 치우친 분포를 먼저 확인한다
SibSp, Parch	동행 형제·배우자, 부모·자녀 수	두 값과 본인을 더해 FamilySize를 만든다
Sex, Embarked	기록된 성별, 승선 항구	범주별 인원·생존율로 비교한다

열	의미	분석할 때 구분할 점
Cabin	객실 정보	빈 칸이 많아 단순 삭제 전에 규모를 확인한다

3.2.2 파일 읽기와 결측 진단

Conda 환경 파일을 내려받아 environment.yml로 저장하고, EDA 실행 파일을 같은 폴더에 저장한다. 그 폴더에서 conda env create -f environment.yml, conda activate titanic-eda를 실행한다. data/에 두 CSV를 넣은 다음 아래 코드를 따라 해 본다.

```
import pandas as pd
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
print(train.shape, test.shape)
print(train.head(3).to_string(index=False))
print(train.isna().sum().sort_values(ascending=False))
```

노션 첨부 파일에서는 train의 생존자가 342명, 전체 생존율이 약 38.4%다. Age는 177행, Cabin은 687행이 비어 있다. 나이 분포를 그리면 나이가 기록된 승객만 포함되므로 그 그림의 분모는 891명이 아니다. 관찰하지 않은 나이를 전부 평균으로 채우면 평균 근처에 실제로 없던 빈도가 생길 수 있다. 이 EDA에서는 원본을 바꾸지 않고 결측 수를 기록한다.

3.2.3 집단별 생존율과 분모

생존율은 Survived=1인 수를 해당 집단 인원으로 나눈 값이다. 객실 등급과 성별을 함께 묶으면 단순 성별 평균 뒤에 숨은 집단 구성을 볼 수 있다.

```
print(train['Survived'].value_counts())
print(train['Survived'].mean())
rates = train.groupby(['Sex', 'Pclass'])['Survived'].agg(['size', 'sum', 'mean'])
print(rates)
```

이 파일에서 여성 1등급은 94명 중 91명이 생존해 약 96.8%이고, 여성 3등급은 144명 중 72명이 생존해 50.0%다. size가 분모, sum이 분자, mean이 생존율이다. 두 집단의 비율 차이는 관찰 결과다. 성별이나 등급이 생존을 직접 일으켰다는 인과 결론은 이 표만으로 내리지 않는다.

3.2.4 시각화와 가족 규모

실습 폴더에서 다음 명령을 실행하면 output/tables/의 결측·집단별 집계표와 output/charts/titanic_eda.png가 생성된다.

```
python titanic_eda.py --train data/train.csv --test data/test.csv --output output
```

차트는 생존 여부별 인원, 성별·등급별 생존율, 나이가 기록된 승객만의 나이 분포, 등급별 요금 분포를 보여준다. 요금 박스플롯에서 이상값 점을 숨겨도 원자료를 삭제한 것은 아니다. FamilySize=SibSp+Parch+1별 집계표도 확인하되, 이는 같은 예약의 가족 관계 전체가 아니라 파일에 기록된 동행자 수의 근사치다.

차트를 직접 다시 그릴 때는 아래 코드를 3.2.2~3의 코드에 이어 실행한다. Age 결측 행은 나이 히스토그램에서 자동 제외되며, 성별·등급별 막대의 높이는 집단별 비율이다.

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style='whitegrid')
fig, axes = plt.subplots(2, 2, figsize=(12, 9), layout='constrained')
sns.countplot(data=train, x='Survived', ax=axes[0, 0], color='#396a94')
```

```
sns.barplot(data=rates.reset_index(), x='Pclass', y='mean', hue='Sex', ax=axes[0, 1])
axes[0, 1].set_ylim(0, 1)
sns.histplot(data=train, x='Age', bins=20, ax=axes[1, 0], color='#287c86')
sns.boxplot(data=train, x='Pclass', y='Fare', ax=axes[1, 1], showfliers=False)
axes[0, 0].set_title('Survival counts')
axes[0, 1].set_title('Survival rate by class and sex')
axes[1, 0].set_title('Recorded ages only')
axes[1, 1].set_title('Fare by class; outlier dots hidden')
fig.savefig('titanic_eda.png', dpi=160)
plt.show()
```

그림 3-15. Titanic train.csv의 EDA 실행 결과 — 왼쪽 아래 나이 분포는 나이가 기록된 승객만 포함하고, 오른쪽 아래 요금 상자그림에서는 점 표시만 숨겼다. 아래 그림은 노션 첨부 파일의 실제 값을 위 파이프라인으로 실행한 결과다.

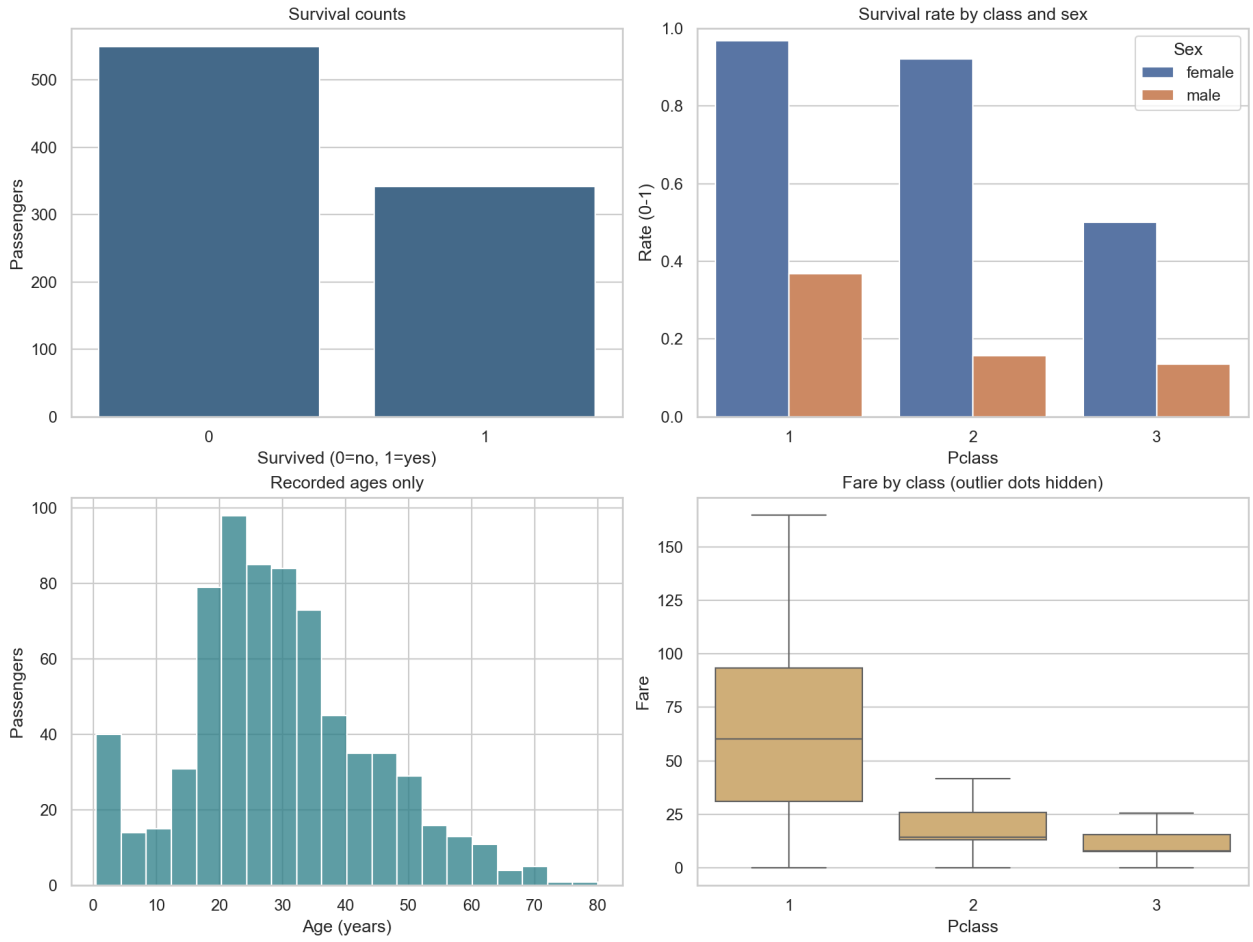


그림 3-15. Titanic 승객의 생존·나이·요금 EDA

노션 원고의 모든 열 히스토그램·일괄 평균·항구 코드 숫자 상관행렬은 이 실습에 넣지 않았다. PassengerId는 식별자이고 Pclass·Embarked는 범주이므로 연속 수치처럼 평균과 상관계수를 해석하면 학생들이 잘못된 결론을 낼 수 있기 때문이다. 예측 모델과 LabelEncoder는 후속 모델링 수업에서 별도로 다룬다.

3.2.5 결과 해석과 제출

titanic-observations.md에 ① 첫 행으로 확인한 관찰 단위, ② 결측이 많은 두 열과 결측률, ③ 성별×등급별 집단 두 개의 인원·생존자·생존율, ④ 차트에서 읽은 사실 두 개, ⑤ 이 데이터만으로는 답할 수 없는 원인 주장 하나를 적는다. 코드와 summary.txt, 집계 CSV 및 PNG를 함께 제출한다. 원본 CSV와 인증 토큰은 제출하지 않는다. 분석이 멈추면 train.csv의 경로·열 이름, Conda 환경 활성화, Survived의 존재 여부부터 확인한다.

3.3 요약

- Kaggle에서 데이터를 찾으면 먼저 한 행의 의미와 질문에 필요한 열이 있는지 확인한다.
- 이 장의 예제는 안내된 netflix_titles.csv를 사용한다. 다른 시점의 파일을 받으면 행 수와 분석 결과가 달라질 수 있다.
- 원본은 별도로 보존하고 열 이름·행 수·키·결측을 먼저 확인한다. 체크섬은 수업 고정본의 파일 버전을 맞출 때만 쓴다.
- 프로파일링으로 문제를 측정한 뒤 정제 규칙을 결정한다.
- 정제는 오류·불일치를 다루고, 전처리는 분석 가능한 구조와 특징을 만든다.
- Movie의 분과 TV Show의 시즌 수를 한 숫자 평균으로 섞지 않는다.
- 다중값 열은 장형 관계 테이블로 정규화한 뒤 집계한다.
- EDA는 질문·지표·분모·차트·결론·한계를 연결하는 반복 과정이다.
- 카탈로그 메타데이터는 시청량·선호·가입·매출·투자·인과를 설명하지 않는다.
- 같은 EDA 순서를 Titanic 승객 데이터에 적용해 집단별 분모와 결측에 따른 해석 차이를 확인한다.

3.4 퀴즈

1. Kaggle의 Dataset과 Code를 올바르게 구분한 설명은 무엇인가?

1) Dataset은 항상 정답 코드이고 Code는 원본 파일이다.

2) Dataset은 데이터와 설명을 제공하고 Code는 그 데이터를 사용하는 실행 문서다.

3) 둘은 라이선스와 버전을 자동 보증한다.

4) Code가 실행되면 데이터 출처 검증은 필요 없다.

정답: 2 해설: 실행 가능한 Notebook과 데이터 파일·설명은 역할이 다르며 각각 입력 버전과 권리를 확인해야 한다.

2. CSV를 처음 열었을 때 분석 전에 우선 확인할 내용은 무엇인가?

1) 파일 아이콘과 폴더 색상

2) 행·열 수, 열 이름, 기본키 중복과 결측

3) 차트 색상과 글꼴

4) 파일명을 더 짧게 바꾸는 일

정답: 2 해설: 분석 코드가 기대한 구조인지 먼저 확인해야 이후 집계와 시각화를 신뢰할 수 있다. 체크섬은 교수자가 배포한 고정본과 완전히 같은 파일인지 확인할 때만 보조적으로 사용한다.

3. Unknown으로 결측을 대체한 결과를 올바르게 설명한 것은 무엇인가?

1) 실제 미상 감독이 새로 관측되었다.

2) 결측이 완전히 해결되어 편향이 사라졌다.

3) 결측을 명시적 범주로 부호화했으므로 분석에서 별도로 해석해야 한다.

4) 원본 데이터의 정확성이 높아졌다.

정답: 3 해설: 대체값은 정보 복원이 아니라 분석을 위한 표현 선택이다.

4. Movie와 TV Show의 duration_value 평균을 합치면 안 되는 이유는 무엇인가?

1) CSV에는 숫자가 없기 때문이다.

2) Movie는 분, TV Show는 시즌 수로 단위가 다르기 때문이다.

3) 모든 값이 중복이기 때문이다.

4) duration이 기본키이기 때문이다.

정답: 2 해설: 단위가 다른 수치를 합친 평균은 해석할 수 없다.

5. 장르 관계 수의 합이 콘텐츠 행 수보다 큰 이유는 무엇인가?

1) 반드시 중복 오류가 있기 때문이다.

2) 한 콘텐츠가 여러 장르에 속할 수 있기 때문이다.

3) 결측을 모두 삭제했기 때문이다.

4) 체크섬이 틀렸기 때문이다.

정답: 2 해설: 다대다 관계를 장형으로 펼치면 한 콘텐츠가 여러 관계 행을 만든다.

6. 2021년 등록 건수가 낮다는 사실만으로 “Netflix가 등록 전략을 축소했다”고 결론 내릴 수 없는 가장 중요한 이유는 무엇인가?

1) title이 문자열이기 때문이다.

2) 2021년이 부분 관측 기간일 수 있고 현재 스냅샷에 삭제 콘텐츠가 없을 수 있기 때문이다.

3) Movie와 TV Show가 모두 있기 때문이다.

4) 막대그래프를 사용했기 때문이다.

정답: 2 해설: 불완전 기간과 스냅샷의 생존 편향을 먼저 검토해야 한다.

7. 상위 장르를 “인기 장르”라고 부르려면 추가로 필요한 데이터는 무엇인가?

1) 파일명

2) 시청·평점·사용자 표본과 측정 설계

3) 열 순서

4) 체크섬만

정답: 2 해설: 카탈로그 빈도는 인기나 선호를 직접 측정하지 않는다.

8. Titanic test.csv만으로 실제 생존율을 계산할 수 없는 이유는 무엇인가?

1) 승객 수가 적기 때문이다.

2) Survived 열이 없기 때문이다.

3) Pclass가 범주형이기 때문이다.

4) Fare에 큰 값이 있기 때문이다.

정답: 2 해설: 실제 생존 여부가 제공되지 않으므로 test 파일만으로 생존율을 구할 수 없다.

9. 여성 1등급 승객 94명 중 91명이 생존했다면 이 집단의 생존율은 얼마인가?

1) 약 9.7%

2) 약 38.4%

3) 약 96.8%

4) 전체 승객 수가 없으므로 계산할 수 없음

정답: 3 해설: 이 집단의 생존자 91명을 집단 인원 94명으로 나눈다.

10. 나이 결측을 전부 평균으로 채운 뒤 히스토그램을 그릴 때 생길 수 있는 문제는 무엇인가?

1) 평균 근처에 관찰하지 않은 값이 인위적으로 쌓인다.

2) 모든 승객의 생존 결과가 사라진다.

3) 객실 등급이 자동 변경된다.

4) 원래 나이 분포가 정확히 복원된다.

정답: 1 해설: 같은 대체값을 여러 행에 넣으면 실제 관찰 분포와 다른 봉우리가 생긴다.

11. 성별·객실 등급별 생존율 차트에서 바로 말할 수 있는 것은 무엇인가?

1) 성별이 생존의 유일한 원인이다.

2) 등급은 생존과 무관하다.

3) 관찰된 집단별 비율이 다르며 원인 판단에는 추가 분석이 필요하다.

4) test.csv의 승객별 실제 생존 결과를 알 수 있다.

정답: 3 해설: 관찰 자료의 집단 차이는 인과관계의 증명이 아니다.

4주차 (데이터 수집·저장·전처리·분석)_넷플릭스

1장: 데이터 분석 개요

- 1-1. 데이터 분석이란?
- 1-2. 데이터 분석 프로세스 이해
- 1-3. 넷플릭스 데이터 소개 및 분석 목표 설정
- 분석 목표 설정하기

2장: 데이터 분석 환경 구축

- 2-1. 데이터 분석 도구 및 환경 소개
- 2-2. Conda 가상환경 구축 및 VS Code 연동
- 2-3. 데이터 수집 및 저장

3장: 데이터 전처리 (Data Preprocessing)

- 3-1. 데이터 정제 (Data Cleaning)
- 3-2. 결측치 처리
- 3-3. 데이터 변환

4장: 탐색적 데이터 분석 (EDA)

- 4-1. 기초 통계량 확인
- 4-2. 주요 피처 분석
- 장르별 작품 수 분석
- 감독별 작품 수 분석
- 연도별 작품 제작 트렌드
- 영화 길이(Duration) 분석
- 시청 등급(Rating) 분포

연습문제 1

- [문제 1] 가장 많이 출연한 배우는 누구일까? (Top 10 Actors)
- [문제 2] TV 쇼는 보통 몇 시즌으로 구성될까?
- [문제 3] 넷플릭스는 몇 월에 콘텐츠를 가장 많이 추가할까?

5장: 데이터 시각화 (Visualization)

연습문제 2

[문제 1] 넷플릭스의 연도별 콘텐츠 추가량과 콘텐츠 투자액(가상 데이터)를 이중 축 차트와 산점도를 사용하여 시각화하세요.

6장: 데이터 분석 심화

6-1. 관계 분석

6-1장 전체 코드 (`6-1.py`)

2. 주요 국가별 선호 장르 분포

6-2. 네트워크 분석

1. 네트워크 그래프(이미지) 분석

2. 중심성 분석 (실행 로그) 분석

6-3. 텍스트 마이닝

워드 클라우드 종합 분석

1. 전체 넷플릭스 콘텐츠 키워드 (상단)

2. 'Dramas' 장르 핵심 키워드 (하단 좌측)

3. 'Comedies' 장르 핵심 키워드 (하단 우측)

장르별 비교 및 핵심 인사이트

6-4. 추천 시스템 기초: 콘텐츠 기반 필터링

6-5. 협업 필터링 심화: 아이템 기반 필터링

핵심 아이디어: "이 영화를 좋아한 사람들은, 다른 어떤 영화를 좋아했을까?"

동작 원리 3단계

장점과 단점

7장: 머신러닝 적용 가능성 탐색

7-1. 회귀(Regression) 모델

7-2 모델 성능 분석 및 고찰

1. 정량적 성능 평가: MAE 변화 분석

2. 정성적 성능 평가: 예측 결과 분석

3. 결론 및 시사점

📌 참고사이트: https://www.kaggle.com/code/iamsouravbanerjee/netflix-is-all-you-need

1장: 데이터 분석 개요

데이터 분석의 여정을 시작하기에 앞서, 우리는 왜 데이터를 분석해야 하는지, 그리고 어떤 과정을 거치게 될 것인지에 대한 큰 그림을 이해해야 합니다. 이 장에서는 데이터 분석의 근본적인 개념을 배우고, 우리가 앞으로 다룰 넷플릭스 데이터를 통해 어떤 질문을 던지고 해결해 나갈지 구체적인 목표를 설정합니다.

1-1. 데이터 분석이란?

- 데이터 분석(Data Analysis) 이란 단순히 숫자를 계산하거나 차트를 그리는 행위를 넘어, **유용한 정보를 발견하고 결론을 도출하며, 궁극적으로 합리적인 의사결정을 지원하기 위해 데이터를 탐색하고 정제, 변환, 모델링하는 일련의 과정**을 의미합니다.

과거에는 경영진의 직감이나 경험에 의존해 중요한 결정을 내리는 경우가 많았습니다. 하지만 오늘날에는 방대한 데이터를 기반으로 객관적인 근거를 찾아내는 '데이터 기반 의사결정(Data-Driven Decision Making)' 이 기업의 핵심 경쟁력으로 자리 잡았습니다.

> 쉽게 비유해 볼까요? 👨‍⚕️
>
> 의사가 환자를 진단하는 과정은 데이터 분석과 매우 유사합니다.
>
> - **데이터 수집:** 환자의 증상, 체온, 혈압 등 다양한 원시 데이터(raw data) 를 수집합니다.
> - **데이터 분석:** 수집된 데이터를 바탕으로 혈액 검사, X-ray 등의 분석을 통해 눈에 보이지 않던 **패턴**이나 이상 징후(information) 를 찾아냅니다.
> - **결론 및 의사결정:** 분석 결과를 종합하여 '폐렴'이라는 통찰(insight) 을 얻고, '항생제 처방'이라는 최종 **의사결정**을 내립니다.

이처럼 데이터 분석은 원석과 같은 데이터 속에서 보석 같은 '인사이트'를 캐내는 과정이며, 넷플릭스와 같은 글로벌 기업들은 이러한 분석을 통해 "어떤 영화를 추천할지", "다음 시즌에는 어떤 드라마를 제작할지"와 같은 중요한 비즈니스 결정을 내리고 있습니다. 📊

1-2. 데이터 분석 프로세스 이해

데이터 분석은 정해진 순서 없이 주먹구구식으로 진행되지 않습니다. 일반적으로 다음과 같은 체계적인 단계를 따르며, 각 단계는 서로 유기적으로 연결되어 상호작용합니다. 대부분의 데이터 분석은 **체계적인 6단계**를 따릅니다:

1. **문제 정의 (Problem Definition)**
    - 분석 목표를 구체적이고 측정 가능하게 정의한다. 모호한 문제 정의는 분석 방향성을 상실시킨다.

    - 예시: "넷플릭스는 어떤 장르의 콘텐츠를 더 많이 제작해야 할까?"

2. **데이터 수집 (Data Collection)**
    - 정의된 문제를 해결하기 위해 필요한 데이터를 모으는 단계입니다. 데이터는 내부 데이터베이스, 외부 API, 설문조사, 웹 크롤링, 공개 데이터셋 등을 통해 필요한 데이터를 수집한다. 데이터의 품질과 관련성을 고려하여 선별한다.
3. **데이터 전처리 및 정제 (Data Preprocessing & Cleaning)**
    - '데이터 분석의 80%는 전처리다'라는 말이 있을 정도로 중요하고 시간이 많이 소요되는 단계입니다. 수집된 원시 데이터에는 오류(오타), 누락된 값(결측치), 일관성 없는 형식 등이 존재하기 마련입니다.
      분석에 적합하도록 데이터를 깨끗하게 다듬는 과정입니다.
4. **탐색적 데이터 분석 (Exploratory Data Analysis, EDA)**
    - 데이터를 본격적으로 파헤치며 숨겨진 패턴이나 관계, 구조를 파악하는 단계입니다. 기초 통계량을 계산하고, 다양한 기법으로 데이터를 시각화하며 데이터에 대한 이해도를 높입니다.
      이 과정에서 새로운 분석 아이디어를 얻거나 초기 가설을 수정하기도 합니다.
    - 데이터의 구조, 분포, 패턴, 변수 간 관계를 탐색한다. 기초 통계량 계산과 시각화를 통해 데이터에 대한 이해를 높인다.
5. **모델링 및 분석 (Modeling & Analysis)**
    - EDA를 통해 얻은 이해를 바탕으로 통계 모델이나 머신러닝 알고리즘을 적용하여 예측, 분류, 군집화 등의 심층 분석을 수행합니다.
      문제의 종류에 따라 적합한 분석 기법을 선택하는 것이 중요합니다.
6. **결과 해석 및 전달 (Interpretation & Communication)**
    - 분석 결과를 해석하여 의미 있는 결론과 '인사이트'를 도출하고, 이를 차트, 보고서, 대시보드 등의 형태로 정리하여 의사결정권자나 팀 동료들이 쉽게 이해할 수 있도록 전달하는 마지막 단계입니다.
      분석 결과는 결국 '스토리텔링'을 통해 그 가치가 빛나게 됩니다.

1-3. 넷플릭스 데이터 소개 및 분석 목표 설정

Kaggle에 공개된 'Netflix Movies and TV Shows' 데이터셋을 사용합니다. 이 데이터셋은 넷플릭스에 등록된 콘텐츠의 다양한 정보를 담고 있으며, 주요 컬럼은 다음과 같습니다.

- `show_id`: 콘텐츠 고유 ID
- `type`: 콘텐츠 유형 (Movie / TV Show)

- `title`: 제목
- `director`: 감독
- `cast`: 출연 배우
- `country`: 제작 국가
- `date_added`: 넷플릭스에 추가된 날짜
- `release_year`: 콘텐츠가 처음 개봉된 연도
- `rating`: 시청 등급
- `duration`: 콘텐츠 길이 (영화: 분, TV쇼: 시즌)
- `listed_in`: 장르
    - `description`: 콘텐츠 요약 설명

분석 목표 설정하기

위 데이터와 분석 프로세스를 바탕으로, 우리는 다음과 같은 질문들에 대한 답을 찾아가는 것을 이번 장의의 **핵심 목표**로 설정합니다.

- **(현황 파악)** 넷플릭스에는 영화와 TV 쇼 중 어떤 유형의 콘텐츠가 더 많은가?
- **(트렌드 분석)** 시간의 흐름에 따라 넷플릭스 콘텐츠는 어떻게 변화해 왔는가? (연도별 제작/추가 트렌드)
- **(주요 특징 분석)** 어떤 국가가 콘텐츠 제작을 주도하고 있는가? 가장 대중적인 장르는 무엇인가?
- **(관계 분석)** 특정 장르와 시청 등급 간에 특별한 관계가 존재하는가?
- **(심화 탐색)** 콘텐츠 설명을 통해 핵심 키워드를 파악할 수 있을까?

위의 질문들을 해결해 나가는 과정을 통해 여러분은 데이터 분석의 전반적인 흐름을 자연스럽게 익히고, 데이터를 통해 의미 있는 이야기를 만들어내는 데이터 분석가로서의 첫걸음을 내딛게 될 것입니다.

2장: 데이터 분석 환경 구축

1장에서 우리는 데이터 분석의 전체적인 그림과 목표를 설정했습니다. 이제는 실제 분석을 수행할 작업장을 마련할 차례입니다. 훌륭한 목수가 좋은 연장을 고르듯, 데이터 분석가에게

는 안정적이고 효율적인 분석 환경이 필수적입니다.

이 장에서는 분석에 필요한 핵심 도구들을 소개하고, **재현 가능하고 독립적인 분석 환경**을 위해 Conda 가상환경을 구축합니다. 마지막으로, 우리 분석의 재료가 될 넷플릭스 데이터를 VS Code 내에서 불러와 확인하는 것으로 2장을 마무리합니다.

2-1. 데이터 분석 도구 및 환경 소개

우리의 분석 여정에 함께할 든든한 도구들을 소개합니다.

- **Python** 🐍: 데이터 분석의 표준 언어입니다. 문법이 간결하여 배우기 쉽고, 강력한 분석용 라이브러리 생태계를 갖추고 있어 전 세계 데이터 과학자들에게 가장 사랑받는 언어입니다.
- **Pandas**: **데이터 전처리 및 탐색(EDA)** 단계의 핵심 도구입니다. 엑셀의 스프레드시트와 유사한 데이터프레임(DataFrame) 이라는 구조를 사용하여, 파이썬으로 데이터를 자유자재로 정제하고, 변형하고, 분석할 수 있게 해줍니다.
- **Numpy**: 파이썬에서 과학적 계산, 특히 대규모 배열과 행렬 연산을 빠르고 효율적으로 처리하도록 돕는 라이브러리입니다. Pandas의 근간을 이루는 중요한 도구입니다.
- **VS Code (Visual Studio Code)** 💻: 우리의 통합 개발 환경(IDE) 입니다. 단순한 텍스트 편집기를 넘어, 강력한 확장 기능을 통해 파이썬 개발과 데이터 분석을 위한 최적의 환경을 제공합니다. 특히 **Jupyter 확장 기능**을 설치하면, VS Code 내에서 직관적인 노트북(`.ipynb`) 파일을 생성하고 실행하며 탐색적 데이터 분석(EDA) 을 효율적으로 수행할 수 있습니다.
- **시각화 도구 (Matplotlib, Seaborn, Plotly)** 📊: **결과 해석 및 전달** 단계에서 우리의 분석을 빛내줄 도구들입니다. Matplotlib은 파이썬 시각화의 기초이며, Seaborn은 Matplotlib을 더 쉽고 아름답게 만들어줍니다. Plotly는 사용자와 상호작용하는 동적인 차트를 만드는 데 특화되어 있습니다.

2-2. Conda 가상환경 구축 및 VS Code 연동

프로젝트마다 사용하는 라이브러리의 버전이 다를 경우 충돌이 발생할 수 있습니다. 이를 방지하기 위해 가상환경(Virtual Environment) 을 사용합니다. 가상환경은 프로젝트별로 독립된 공간을 만들어주는 '디지털 격리실'과 같습니다.

1. 가상환경 생성 및 라이브러리 설치

Anaconda Prompt (Windows) 또는 터미널(macOS)을 열고, 이전과 동일하게 netflix 가상환경을 만들고 필요한 라이브러리를 설치합니다.

```
# 1. 'netflix' 가상환경 생성 (python 3.9 버전으로)
conda create -n netflix python=3.9

# 2. 'netflix' 가상환경 활성화
conda activate netflix

# 3. (netflix) 환경에 핵심 라이브러리 설치
pip install pandas numpy matplotlib seaborn plotly wordclou
d notebook
```

2. VS Code 설정 및 실행

이제 VS Code에서 이 netflix 환경을 사용하도록 설정합니다.

- **VS Code 실행**: VS Code를 엽니다.
- **확장 기능 설치**: 좌측 메뉴에서 확장(Extensions) 탭을 클릭하고, `Python`과 `Jupyter`를 검색하여 Microsoft에서 제공하는 공식 확장 기능을 각각 설치합니다.
- **프로젝트 폴더 열기**: `File > Open Folder...`를 클릭하여 넷플릭스 데이터셋(`.csv` 파일)이 저장된 폴더를 엽니다.
- **새 노트북 파일 또는 Python 파일 생성**: `File > New File...`을 클릭한 후, `Jupyter Notebook`을 선택하여 `.ipynb` 확장자를 가진 새 분석 노트 또는 코드를 만듭니다.N
- **Python 인터프리터 선택**: 우측 상단의 `Select Kernel` 버튼을 클릭한 뒤, `Python Environments...`를 선택하고 목록에서 우리가 만든 `conda: 'netflix'` 환경을 선택합니다. 이제 이 노트북 파일은 우리가 설정한 `netflix` 가상환경에서 실행됩니다.

2-3. 데이터 수집 및 저장

이제 **데이터 분석 프로세스**의 **'데이터 수집'** 단계를 VS Code 환경에서 실제로 수행해 보겠습니다.

[실습 코드]

```
# 1. 분석에 필요한 모든 라이브러리 불러오기
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from wordcloud import WordCloud, STOPWORDS
```

```
import warnings

# 파일 다운로드 및 경로 관리를 위한 라이브러리
import os
import urllib.request

# 불필요한 경고 메시지 숨기기
warnings.filterwarnings("ignore")

print("라이브러리 임포트 완료!")

# 2. 데이터 다운로드 및 수집
# 데이터가 저장될 파일명과 다운로드할 URL을 변수로 지정합니다.
file_name = "netflix_titles.csv"
# (수정) 안정적인 새 URL로 변경했습니다.
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"

# 현재 폴더에 파일이 없으면 URL에서 다운로드합니다.
# os.path.exists()는 파일 존재 여부를 확인하는 함수입니다.
if not os.path.exists(file_name):
    print("데이터셋 다운로드를 시작합니다...")
    urllib.request.urlretrieve(url, file_name)
    print("다운로드가 완료되었습니다.")
else:
    print("데이터셋이 이미 존재합니다.")

# 3. Pandas로 데이터 불러오기
# 다운로드한 CSV 파일을 Pandas의 read_csv 함수를 사용하여 데이터프
레임으로 불러옵니다.
df = pd.read_csv(file_name)

print(f"\n'{file_name}' 파일을 성공적으로 불러왔습니다.")

# 4. 데이터의 첫인상 확인하기 (탐색적 데이터 분석의 시작)
# 4-1. df.head(): 데이터의 최상단 5개 행을 출력하여 구조를 파악합
니다.
print("\n### 1. 데이터 미리보기 (상위 5개 행) ###")
```

```
df.head()

# 4-2. df.shape: 데이터의 크기(행, 열)를 확인하여 전체적인 규모를
파악합니다.
print(f"\n### 2. 데이터 규모 확인 ###")
print(f"이 데이터는 총 {df.shape[0]}개의 콘텐츠 정보와 {df.shape
[1]}개의 속성으로 이루어져 있습니다.")

# 4-3. df.info(): 데이터의 기술적 요약 정보를 확인하여 결측치 존재
여부와 데이터 타입을 파악합니다.
# 이는 다음 장에서 진행할 '데이터 전처리'의 중요한 단서가 됩니다.
print("\n### 3. 데이터 요약 정보 확인 ###")
df.info()
```

3장: 데이터 전처리 (Data Preprocessing)

2장에서 우리는 7,787개의 콘텐츠 정보를 담은 데이터를 성공적으로 불러왔습니다.

`df.info()`를 통해 확인했듯이, 우리의 데이터는 분석을 수행하기에 아직 완벽하지 않으며 `director`, `cast` 등의 컬럼에는 정보가 비어있는 '결측치'가 존재하고, `date_added`와 같은 날짜 정보는 단순 텍스트(`object`)로 저장되어 있습니다.

요리사가 좋은 요리를 위해 재료를 손질하듯, 우리도 분석이라는 요리를 위해 데이터를 깨끗하고 쓰기 좋게 다듬어야 합니다.

이 **데이터 전처리** 과정은 분석 결과의 신뢰도를 결정하는 매우 중요한 단계입니다. 이 장에서는 **결측치를 처리**하고, **데이터를 분석 목적에 맞게 변환**하는 실질적인 기술을 배웁니다.

3-1. 데이터 정제 (Data Cleaning)

데이터 정제는 분석 결과의 신뢰도를 높이기 위한 필수 과정입니다. 잘못된 데이터나 불필요한 데이터가 섞여 있으면 분석 결과가 왜곡될 수 있기 때문입니다.

이번 실습에서는 결측치 처리에 집중하지만, 중복 데이터 제거, 오타 수정 등도 정제의 중요한 일부입니다.

3-2. 결측치 처리

결측치(Missing Value)는 데이터가 누락된 부분을 의미하며, Pandas에서는 `NaN` 으로 표시됩니다.

2장에서 확인한 우리 데이터의 현황은 다음과 같습니다.

- `director`: 2,389개 결측치
- `cast`: 718개 결측치
- `country`: 507개 결측치
- `date_added`: 10개 결측치
- `rating`: 7개 결측치

결측치를 처리하는 방법은 크게 **제거**와 **대체**로 나뉩니다. 우리는 결측치 개수에 따라 다른 전략을 사용할 것입니다.

[실습 코드]

```
import pandas as pd

# 2장에서 생성한 df를 사용하거나, 없다면 파일을 다시 불러옵니다.
try:
    df
except NameError:
    df = pd.read_csv('netflix_titles.csv')


# 1. 컬럼별 결측치 개수 확인
print("### 1. 처리 전 결측치 현황 ###")
print(df.isnull().sum())


# 2. 결측치 처리 (수정된 코드)

# [수정] FutureWarning를 방지하는 방식으로 결측치를 'Unknown'으로 대체
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')

# 결측치가 매우 적은 행 제거
```

```
df.dropna(subset=['date_added', 'rating'], inplace=True)


# 처리 결과 확인
print("\n### 2. 처리 후 결측치 현황 ###")
print(df.isnull().sum())
print(f"\n처리 후 데이터 크기: {df.shape}")
```



3-3. 데이터 변환

데이터 변환은 기존 데이터를 분석 목적에 더 적합한 형태로 가공하는 과정입니다.

예를 들어, `date_added` 컬럼을 실제 날짜 타입으로 변환하면 시간 기반의 분석(연도별, 월별)이 가능해집니다.

[실습 코드]

```
# 3. 데이터 변환 (수정된 코드)

# [수정] ValueError를 방지하기 위해 날짜 변환 전, 앞뒤 공백 제거
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])

# 파생변수 생성
df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.month

# 텍스트 데이터 분리 및 변환
df['duration_value'] = df['duration'].apply(lambda x: int
(x.split()[0]))
# 'Seasons' -> 'Season'으로 단위를 통일하여 정돈
df['duration_unit'] = df['duration'].apply(lambda x: x.spli
t()[1].replace('s',''))

# 4. 변환 결과 확인
print("\n### 3. 데이터 변환 후 요약 정보 ###")
df.info()

print("\n### 4. 변환 후 데이터 미리보기 ###")
# head()는 Jupyter 환경에서 보기 좋게 출력되지만, .py 파일 실행
시에는 print()로 감싸주는 것이 좋습니다.
print(df[['title', 'date_added', 'added_year', 'duration',
'duration_value', 'duration_unit']].head())
```

```
### 3. 데이터 변환 후 요약 정보 ###
<class 'pandas.core.frame.DataFrame'>
Index: 7770 entries, 0 to 7786
Data columns (total 16 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   show_id         7770 non-null   object
 1   type            7770 non-null   object
 2   title           7770 non-null   object
 3   director        7770 non-null   object
 4   cast            7770 non-null   object
 5   country         7770 non-null   object
 6   date_added      7770 non-null   datetime64[ns]
 7   release_year    7770 non-null   int64
 8   rating          7770 non-null   object
 9   duration        7770 non-null   object
 10  listed_in       7770 non-null   object
 11  description     7770 non-null   object
 12  added_year      7770 non-null   int32
 13  added_month     7770 non-null   int32
 14  duration_value  7770 non-null   int64
 15  duration_unit   7770 non-null   object
dtypes: datetime64[ns](1), int32(2), int64(2), object(11)
memory usage: 971.2+ KB

### 4. 변환 후 데이터 미리보기 ###
   title date_added  added_year   duration  duration_value duration_unit
0     3% 2020-08-14        2020  4 Seasons               4         Seaon
1   7:19 2016-12-23        2016     93 min              93           min
2  23:59 2018-12-20        2018     78 min              78           min
3      9 2017-11-16        2017     80 min              80           min
4     21 2020-01-01        2020    123 min             123           min
(netflix) jwon@Mac week3 %
```

👌 이제 우리의 데이터는 한층 더 깨끗하고 정돈되었습니다. 결측치가 모두 처리되었고, 분석에 용이한 새로운 컬럼들도 추가되었습니다.
다음 4장에서는 잘 손질된 이 데이터를 가지고 본격적으로 숨겨진 의미를 찾아 떠나는 '탐색적 데이터 분석(EDA)' 을 시작합니다.

4장: 탐색적 데이터 분석 (EDA)

3장에서 우리는 원석 같았던 데이터를 분석에 적합한 보석으로 가공했습니다.

이제 이 보석이 어떤 색과 모양을 가졌는지, 어떤 가치를 품고 있는지 본격적으로 탐색할 시간입니다.

이 과정을 탐색적 데이터 분석(Exploratory Data Analysis, EDA) 이라고 합니다.

EDA는 정해진 길을 따라가는 것이 아니라, 데이터와 자유롭게 대화하며 숨겨진 패턴, 특이점, 변수 간의 관계를 찾아내는 과정입니다.

다양한 질문을 던지고, 데이터를 요약하고, 시각화를 시도하며 데이터에 대한 깊은 이해를 얻는 것이 목표입니다.

이 과정에서 얻은 통찰은 5장 이후의 시각화와 분석의 방향을 결정하는 중요한 나침반이 됩니다.

4-1. 기초 통계량 확인

데이터를 깊이 파고들기 전, 가장 먼저 데이터의 전반적인 특징을 요약된 숫자로 파악해야 합니다.

이를 기술 통계(Descriptive Statistics) 라고 합니다. Pandas는 숫자형 데이터와 문자형(범주형) 데이터를 요약하는 편리한 기능을 제공합니다.

- **숫자형 데이터**: `release_year`, `duration_value` 등 계산이 가능한 데이터입니다.
  평균, 중앙값, 표준편차 등을 통해 데이터의 중심 경향과 흩어진 정도를 파악할 수 있습니다.
- **문자형 데이터**: `type`, `rating` 등 종류를 나타내는 데이터입니다.
  고유한 값의 개수, 가장 자주 나타나는 값 등을 통해 데이터의 분포를 파악합니다.

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========


# 1-1. 라이브러리 임포트
import pandas as pd
import os
import urllib.request

# 1-2. 데이터 다운로드 및 로드
file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
```

```
if not os.path.exists(file_name):
    print("데이터셋을 다운로드합니다...")
    urllib.request.urlretrieve(url, file_name)
    print("다운로드가 완료되었습니다.")
else:
    print("데이터셋이 이미 존재하므로 다운로드를 건너뜁니다.")

df = pd.read_csv(file_name)

# 1-3. 데이터 전처리 (3장 요약)
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
df.dropna(subset=['date_added', 'rating'], inplace=True)
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])
df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.month
df['duration_value'] = df['duration'].apply(lambda x: int
(x.split()[0]))
df['duration_unit'] = df['duration'].apply(lambda x: x.spli
t()[1].replace('s',''))

print("\n데이터 준비 완료. EDA를 시작합니다.")
print("-" * 40)


# ============================================================
==========
#               2. 탐색적 데이터 분석 (EDA) - 4장
# ============================================================
==========

### 4-1. 기초 통계량 확인
print("### 1. 숫자형 데이터 기술 통계 ###")
print(df.describe())
```

```
print("\n### 2. 문자형 데이터 기술 통계 ###")
print(df.describe(include='object'))
```

4-2. 주요 피처 분석

전체적인 윤곽을 파악했으니, 이제 1장에서 세웠던 질문들을 바탕으로 개별 특징(피처)들을 하나씩 깊이 있게 분석해 보겠습니다.

장르별 작품 수 분석

`listed_in` 컬럼은 여러 장르가 쉼표(,)로 묶여 있습니다. 정확한 장르별 빈도를 알기 위해, 이를 각각 분리하여 분석해야 합니다.

```
### 4-2. 주요 피처 분석
# 장르별 작품 수 분석
genres = df['listed_in'].str.split(',
').explode()
print("\n### 3. 장르별 콘텐츠 수 (Top 10) ###")
print(genres.value_counts().head(10))
```

감독별 작품 수 분석

어떤 감독이 넷플릭스에 가장 많은 작품을 올렸을까요? 전처리 과정에서 'Unknown'으로 대체한 값을 제외하고 분석합니다.

```
# 감독별 작품 수 분석
known_directors = df[df['director'] != 'Unknown']
print("\n### 4. 감독별 작품 수 (Top 10) ###")
print(known_directors['director'].value_counts().head(10))
```

연도별 작품 제작 트렌드

시간에 따라 콘텐츠 제작은 어떻게 변해왔을까요? `release_year`를 기준으로 연도별 제작 편수를 확인합니다.

```
# 연도별 작품 제작 트렌드
print("\n### 5. 출시 연도별 콘텐츠 제작 수 (최근 15년) ###")
print(df['release_year'].value_counts().sort_index(ascending=False).head(15))
```

영화 길이(Duration) 분석

`duration_value`는 영화의 '상영 시간(분)'과 TV 쇼의 '시즌 수'가 섞여 있습니다. 정확한 분석을 위해, **영화(Movie)만** 필터링하여 상영 시간의 통계를 살펴봅니다.

```
# 영화 길이(Duration) 분석
movies_df = df[df['type'] == 'Movie'].copy()
print("\n### 6. 영화 상영 시간(분) 통계 ###")
print(movies_df['duration_value'].describe())

### TV show 에 대한 길이(Run-time) 분석 ###
### 조별 과제 ###
```

시청 등급(Rating) 분포

넷플릭스 콘텐츠는 어떤 시청 등급이 가장 많을까요? `rating` 컬럼의 분포를 확인합니다.

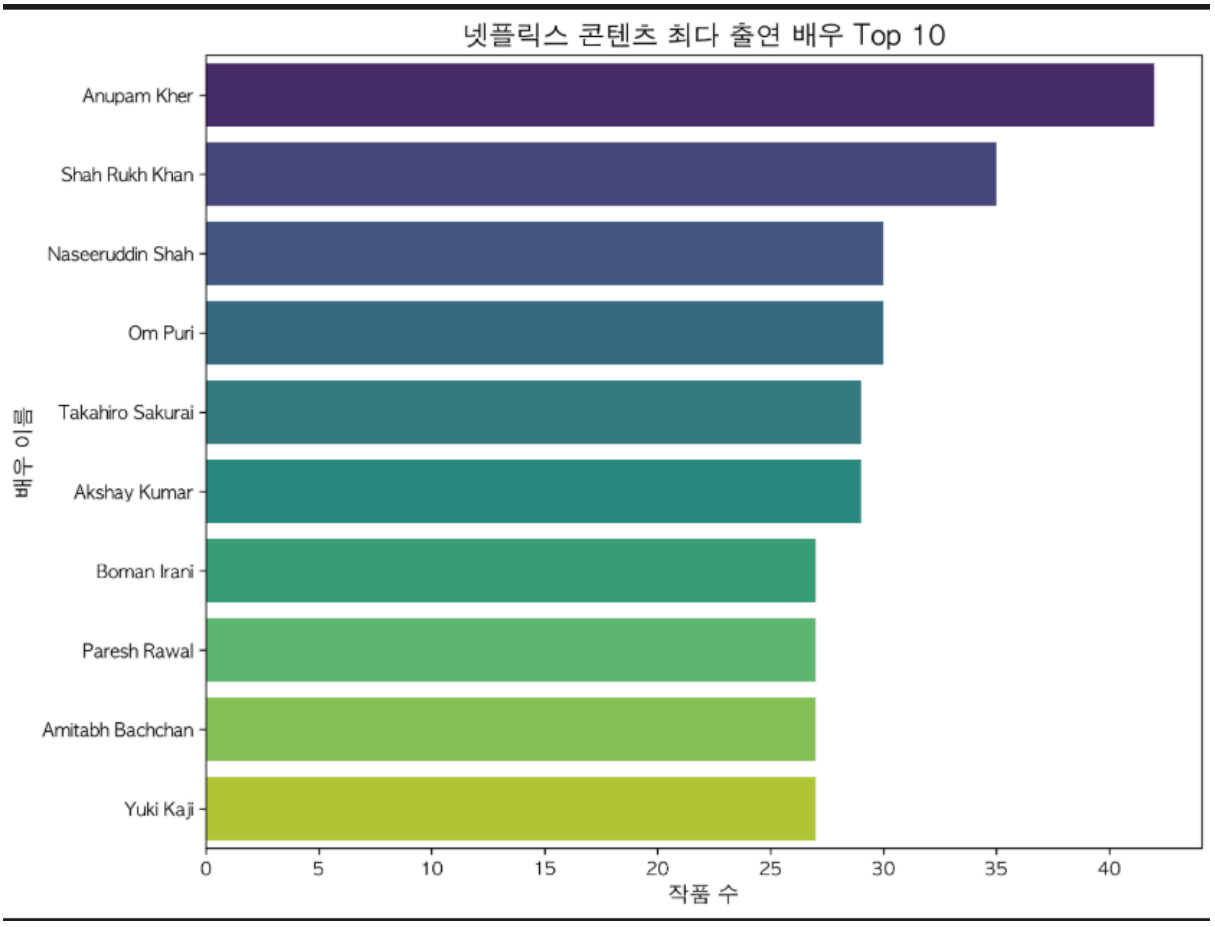
```
# 시청 등급(Rating) 분포
print("\n### 7. 시청 등급별 콘텐츠 수 (Top 10) ###")
print(df['rating'].value_counts().head(10))
```

연습문제 1

[문제 1] 가장 많이 출연한 배우는 누구일까? (Top 10 Actors)

`cast` 컬럼에는 여러 배우의 이름이 쉼표로 구분되어 있습니다. 이 데이터를 활용하여 넷플릭스 콘텐츠에 가장 많이 출연한 배우 Top 10을 찾아보세요.

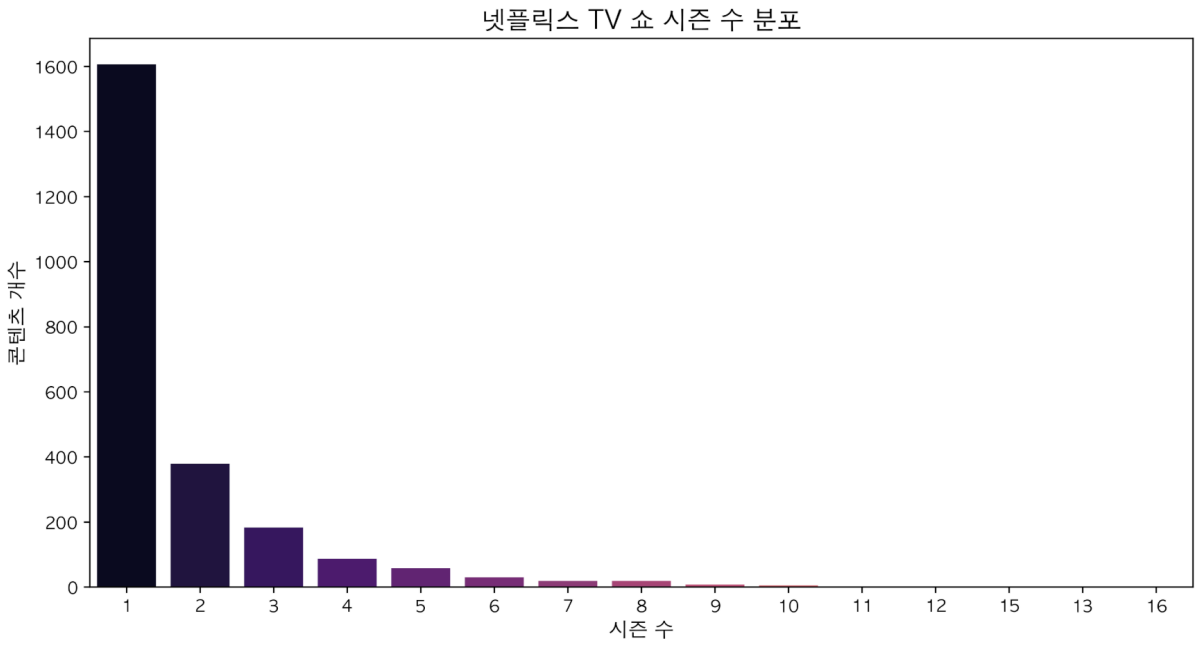
- **힌트 1**: `director` 분석과 마찬가지로 'Unknown' 값은 제외하고 분석하세요.
- **힌트 2**: `listed_in`(장르) 분석처럼, 쉼표와 공백(`', '`)을 기준으로 문자열을 분리하고 `explode()` 함수를 사용해야 합니다.



[문제 2] TV 쇼는 보통 몇 시즌으로 구성될까?

영화의 평균 상영 시간을 분석했던 것처럼, 이번에는 **TV 쇼(TV Show)만** 필터링하여 시즌(`duration_value`)이 일반적으로 어떻게 분포하는지 기초 통계량을 통해 확인해 보세요.

- **힌트**: 먼저 `df['type']`이 'TV Show'인 데이터만 선택하여 새로운 변수에 저장하세요. 그 후 `.describe()` 함수를 사용합니다.

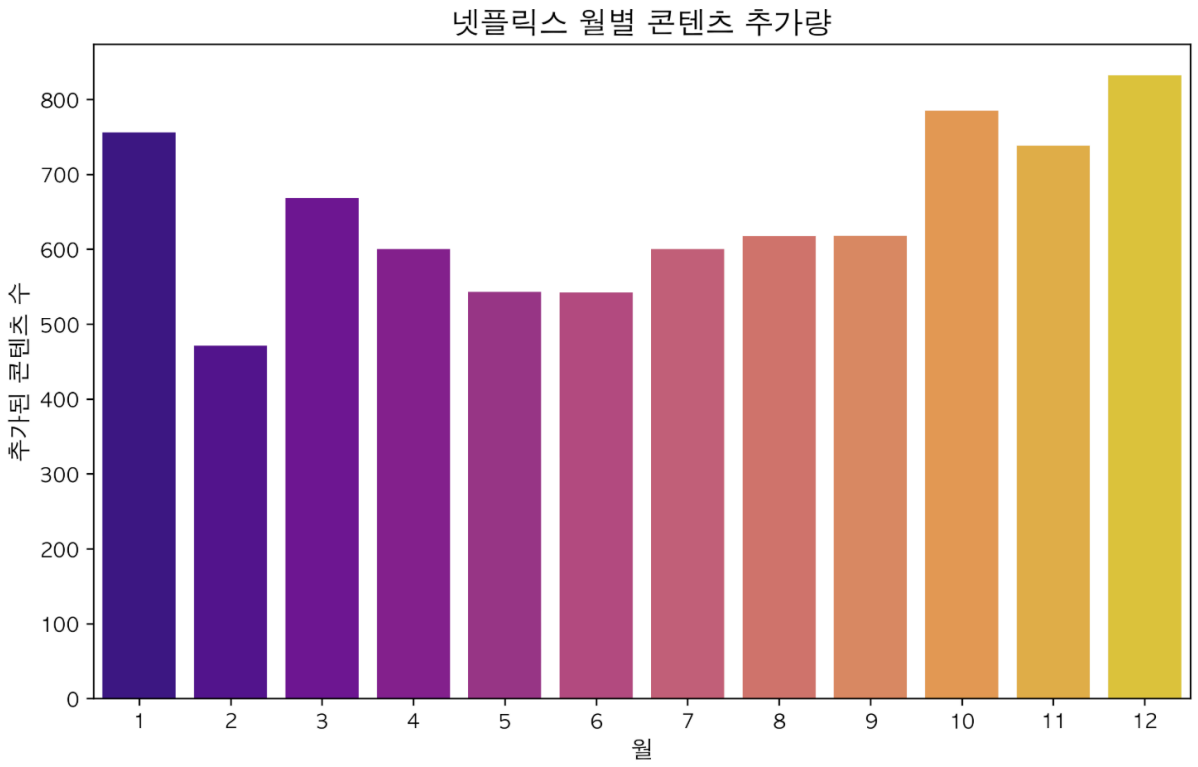


[문제 3] 넷플릭스는 몇 월에 콘텐츠를 가장 많이 추가할까?

우리는 3장에서 `date_added` 컬럼으로부터 `added_month`라는 파생변수를 만들었습니다.

이 컬럼을 사용하여 넷플릭스가 연중 어느 달에 콘텐츠를 가장 활발하게 추가하는지 분석해 보세요.

- **힌트**: `added_month` 컬럼에 `.value_counts()`를 적용한 후, 월 순서대로 보기 좋게 `.sort_index()`로 정렬해 보세요.



5장: 데이터 시각화 (Visualization)

4장까지 우리는 탐색적 데이터 분석(EDA)을 통해 데이터의 구조와 특징을 숫자로 요약하고 파악했습니다.

데이터 시각화는 이 과정에서 발견한 인사이트나 패턴을 **차트와 그래프를 통해 직관적이고 이해하기 쉬운 이야기로 바꾸는 과정**입니다.

"A picture is worth a thousand rows of data."라는 말처럼, 잘 만들어진 시각화 자료 하나는 복잡한 데이터 테이블보다 훨씬 더 강력한 설득력과 통찰력을 제공합니다. 이 장에서는 분석 목적에 맞는 시각화 기법을 선택하고, `Matplotlib`과 `Seaborn` 라이브러리를 활용하여 단일 변수, 다변량 관계, 시간의 흐름에 따른 변화를 효과적으로 표현하는 방법을 배웁니다.

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========
```

```python
# 1-1. 라이브러리 임포트
import pandas as pd
import os
import urllib.request
import matplotlib.pyplot as plt
import seaborn as sns
import platform

# 1-2. 한글 폰트 설정
if platform.system() == 'Darwin':
    plt.rc('font', family='AppleGothic')
elif platform.system() == 'Windows':
    plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)

# 1-3. 차트 저장 폴더 생성
output_dir = 'results'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# 1-4. 데이터 다운로드 및 로드
file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
if not os.path.exists(file_name):
    urllib.request.urlretrieve(url, file_name)
df = pd.read_csv(file_name)

# 1-5. 데이터 전처리 (3장 요약)
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
df.dropna(subset=['date_added', 'rating'], inplace=True)
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])
df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.month
```

```
df['duration_value'] = df['duration'].apply(lambda x: int
(x.split()[0]))
df['duration_unit'] = df['duration'].apply(lambda x: x.spli
t()[1].replace('s',''))

print("데이터 준비 완료. 5장 시각화를 시작합니다.")
print("=" * 50)
```

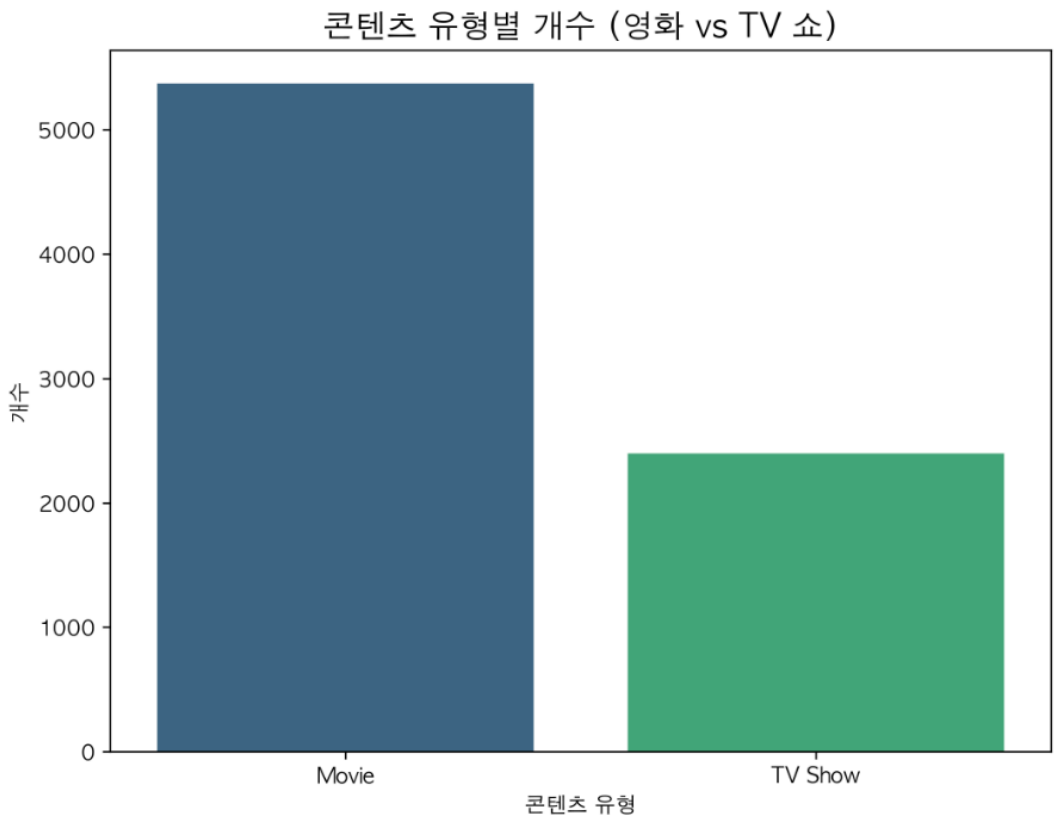
```
### 5-1. 단일 변수 시각화
# 하나의 변수가 가진 데이터의 분포나 구성을 파악하기 위한 시각화입니
다.

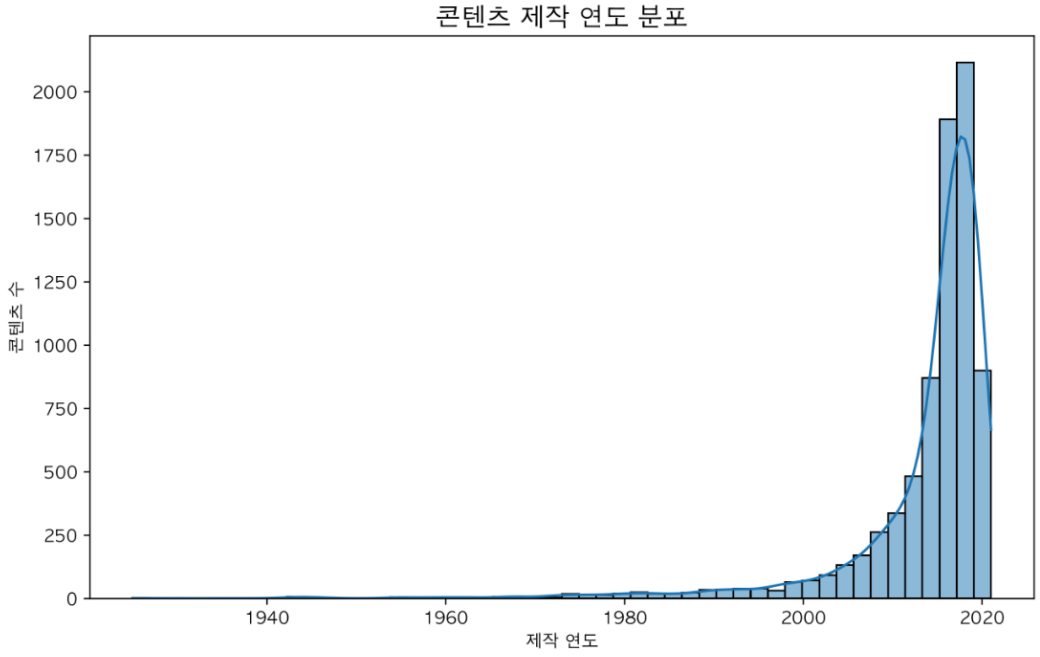
# --- 1) 히스토그램 (Histogram): 숫자형 데이터의 분포 확인 ---
# 영화가 제작된 연도(release_year)의 분포를 히스토그램으로 확인합니
다.
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='release_year', bins=50, kde=True)
plt.title('콘텐츠 제작 연도 분포', fontsize=15)
plt.xlabel('제작 연도')
plt.ylabel('콘텐츠 수')
save_path = os.path.join(output_dir, '5_1_histogram_release
_year.png')
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")

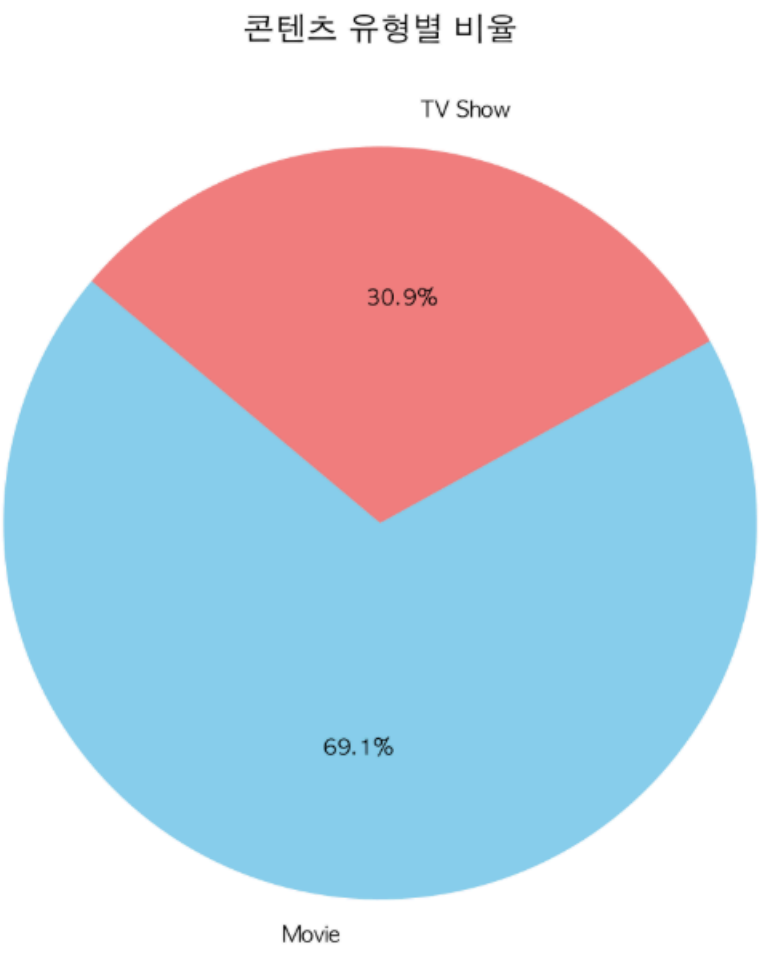

# --- 2) 막대그래프 (Bar Chart): 범주형 데이터의 빈도 비교 ---
# 콘텐츠 유형(type)에 따라 영화와 TV 쇼의 개수를 비교합니다.
plt.figure(figsize=(8, 6))
sns.countplot(data=df, x='type', palette='viridis', order=d
f['type'].value_counts().index)
plt.title('콘텐츠 유형별 개수 (영화 vs TV 쇼)', fontsize=15)
plt.xlabel('콘텐츠 유형')
plt.ylabel('개수')
save_path = os.path.join(output_dir, '5_1_barchart_type.pn
g')
```

```
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")


# --- 3) 파이 차트 (Pie Chart): 범주형 데이터의 비율 확인 ---
# 영화와 TV 쇼가 전체에서 차지하는 비율을 파이 차트로 확인합니다.
plt.figure(figsize=(8, 8))
type_counts = df['type'].value_counts()
plt.pie(type_counts, labels=type_counts.index, autopct='%1.
1f%%', startangle=140, colors=['skyblue', 'lightcoral'])
plt.title('콘텐츠 유형별 비율', fontsize=15)
plt.ylabel('') # 불필요한 y축 라벨 제거
save_path = os.path.join(output_dir, '5_1_piechart_type.pn
g')
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")
print("-" * 50)
```


콘텐츠 유형별 개수 (영화 vs TV 쇼)
5000
4000
3000
2000
1000
0
개수
Movie
TV Show
콘텐츠 유형


콘텐츠 제작 연도 분포
2000
1750
1500
1250
1000
750
500
250
0
콘텐츠 수
1940
1960
1980
2000
2020
제작 연도




```
### 5-2. 다변량 분석 시각화
# 두 개 이상의 변수 간의 관계를 파악하기 위한 시각화입니다.

# --- 1) 산점도 (Scatter Plot): 두 숫자형 변수 간의 관계 확인 ---
# 콘텐츠 제작 연도(release_year)와 넷플릭스 추가 연도(added_year) 사이의 관계를 살펴봅니다.
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='release_year', y='added_year', alpha=0.5)
```

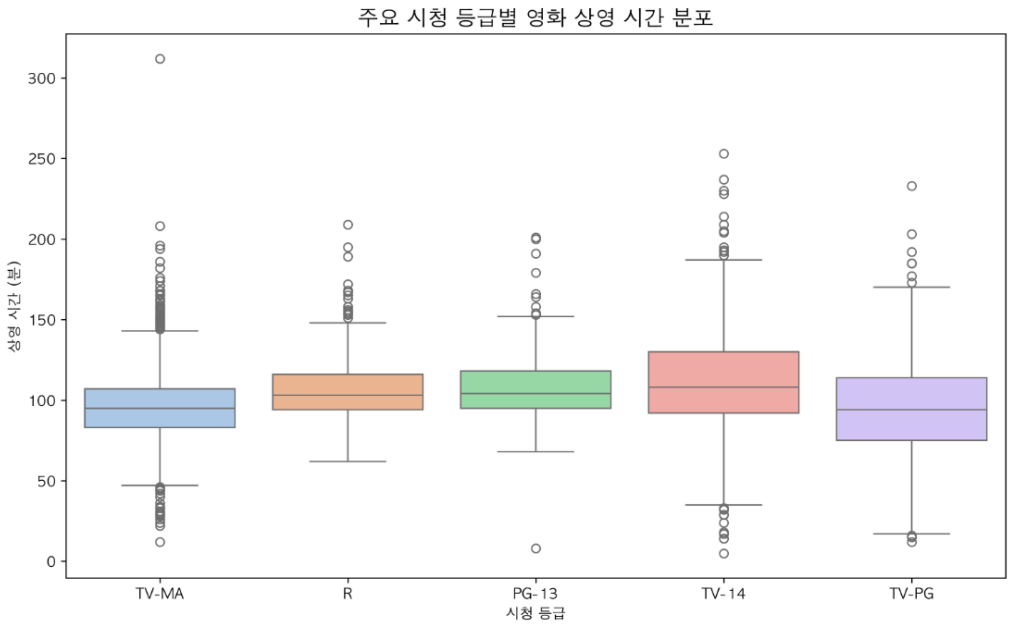
```
plt.title('콘텐츠 제작 연도와 넷플릭스 추가 연도 관계', fontsize=
15)
plt.xlabel('제작 연도')
plt.ylabel('넷플릭스 추가 연도')
save_path = os.path.join(output_dir, '5_2_scatterplot_relea
se_added.png')
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")


# --- 2) 박스 플롯 (Box Plot): 범주에 따른 숫자형 데이터 분포 비
교 ---
# 가장 흔한 5개 시청 등급별로 영화 상영 시간의 분포가 어떻게 다른지
비교합니다.
top_5_ratings = df['rating'].value_counts().head(5).index
movies_top_ratings_df = df[(df['type'] == 'Movie') & (df['r
ating'].isin(top_5_ratings))]

plt.figure(figsize=(12, 7))
sns.boxplot(data=movies_top_ratings_df, x='rating', y='dura
tion_value', palette='pastel')
plt.title('주요 시청 등급별 영화 상영 시간 분포', fontsize=15)
plt.xlabel('시청 등급')
plt.ylabel('상영 시간 (분)')
save_path = os.path.join(output_dir, '5_2_boxplot_rating_du
ration.png')
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")


# --- 3) 히트맵 (Heatmap): 매트릭스 데이터 시각화 ---
# 연도와 월에 따라 콘텐츠가 얼마나 추가되었는지 히트맵으로 확인합니
다.
heatmap_data = pd.crosstab(df['added_year'], df['added_mont
h'])
plt.figure(figsize=(12, 8))
```

```
sns.heatmap(heatmap_data, cmap='YlGnBu', annot=True, fmt
='d', linewidths=.5)
plt.title('연도 및 월별 콘텐츠 추가량', fontsize=15)
plt.xlabel('월')
plt.ylabel('연도')
save_path = os.path.join(output_dir, '5_2_heatmap_year_mont
h.png')
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")
print("-" * 50)
```

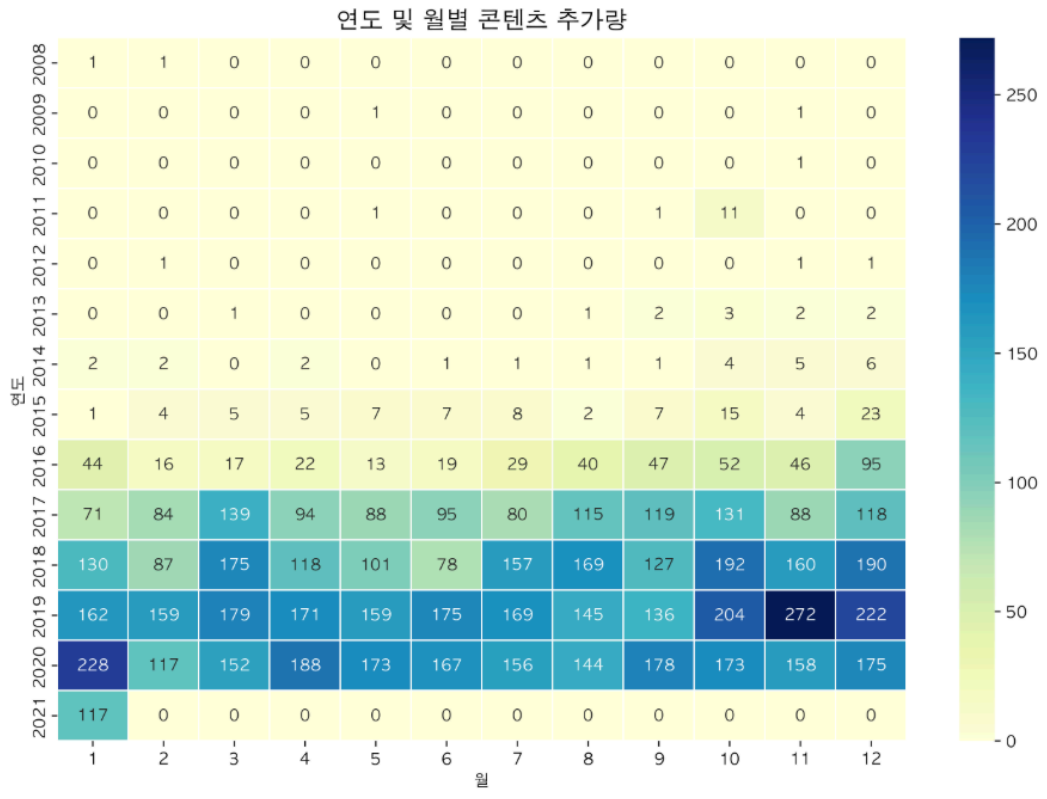


> 이 박스 플롯은 **시청 등급이라는 범주에 따라 영화의 상영 시간(러닝 타임) 분포가 어떻게 다른지** 비교하여 보여줍니다.
>
> - **X축 (시청 등급)**: 넷플릭스에서 가장 빈도가 높은 5개의 시청 등급(`TV-MA`, `R`, `PG-13`, `TV-14`, `TV-PG`)을 나타냅니다.
> - **Y축 (상영 시간)**: 각 영화의 상영 시간(분)을 의미합니다.

> **차트 분석 및 추이**
>
> - **중앙값 (상자 안의 선)**: 모든 등급에서 영화 상영 시간의 중앙값은 약 90분에서 110분 사이에 분포해 큰 차이는 없습니다. 이는 일반적인 상업 영화의 평균 길이를 반영합니

다.

- **분포 범위 (상자의 길이)**: `TV-14` 등급의 상자(Box)가 다른 등급에 비해 길쭉한 편입니다. 이는 `TV-14` 등급 영화들의 상영 시간이 다른 등급에 비해 더 다양하게 분포(상영 시간이 짧은 것부터 긴 것까지 편차가 큼)함을 의미합니다. 반면 `PG-13`은 상자가 짧아 비교적 일정한 상영 시간을 가진 영화들이 많음을 알 수 있습니다.
- **이상치 (점)**: 각 등급의 상자 위아래로 퍼져 있는 점들은 '이상치(Outlier)'로, 평균적인 길이를 크게 벗어나는 매우 길거나 짧은 영화들을 의미합니다. 모든 등급에 이상치가 존재하며, 특히 `TV-MA` 등급에는 300분이 넘는 압도적으로 긴 영화가 존재함을 확인할 수 있습니다.

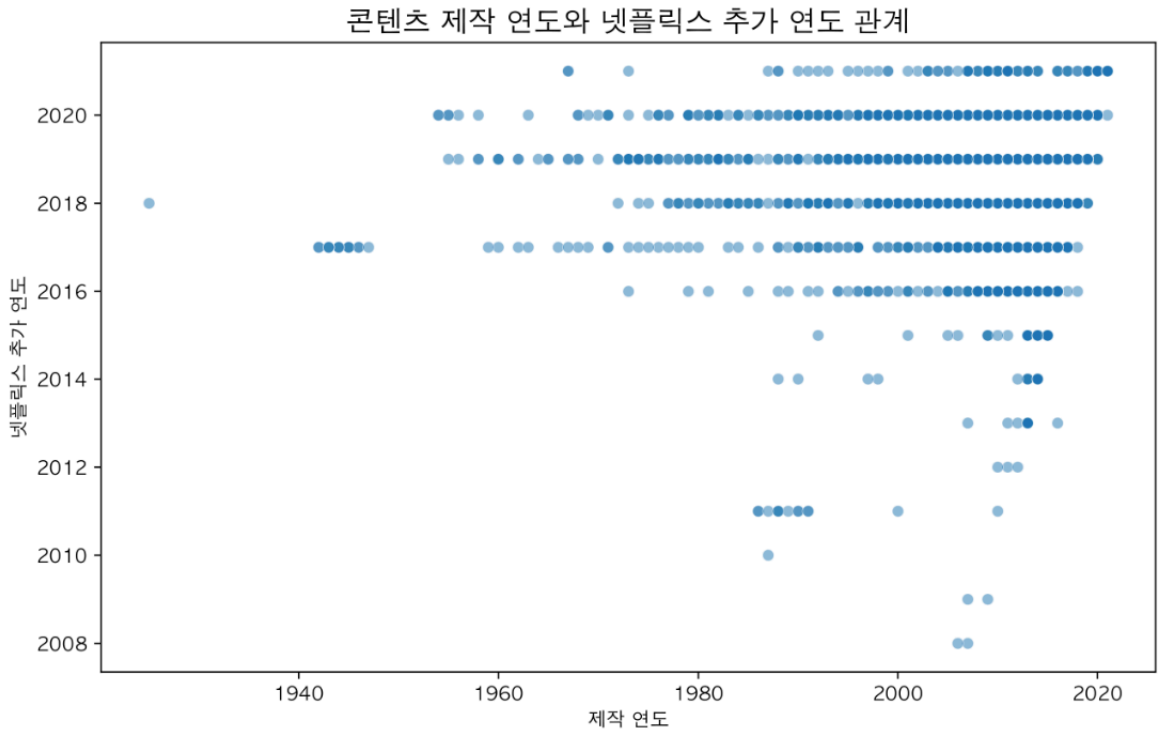


이 히트맵은 **연도와 월이라는 두 가지 기준에 따라 콘텐츠 추가량이 어떻게 분포하는지** 색상의 농도로 시각화합니다.

- **X축 (월)**: 1월부터 12월까지의 월을 나타냅니다.
- **Y축 (연도)**: 2008년부터 2021년까지의 연도를 나타냅니다.
- **값 (색상과 숫자)**: 각 셀의 **색상이 진할수록 더 많은 콘텐츠가 추가**되었음을 의미합니다. 셀 안의 숫자는 해당 연도, 해당 월에 추가된 정확한 콘텐츠 개수입니다. 오른쪽의 컬러바가 색상과 숫자 사이의 척도를 보여줍니다.

차트 분석 및 추이

- **수직적 추이 (연도별)**: 위에서 아래로 내려갈수록 셀의 색상이 전반적으로 진해지는 것을 볼 수 있습니다. 이는 앞서 라인 차트에서 확인했던 것처럼, 시간이 지남에 따라 넷플릭스의 연간 콘텐츠 추가량이 급격히 증가했음을 다시 한번 보여줍니다.
- **수평적 추이 (월별)**: 최근 연도(2018-2020년)의 행을 수평으로 살펴보면, 특정 월에만 집중되기보다는 연중 꾸준히 콘텐츠가 추가되는 경향을 보입니다. 하지만 상대적으로 **하반기(7월~12월)에 색이 더 진한 경향**이 있는데, 이는 연말이나 휴가 시즌을 겨냥한 콘텐츠 공개 전략이 있을 수 있음을 시사합니다.



이 산점도는 **각 콘텐츠가 처음 제작된 연도와 넷플릭스에 추가된 연도 사이의 관계**를 보여주며, 넷플릭스의 콘텐츠 수급 패턴에 대한 중요한 통찰을 줍니다.

- **X축 (제작 연도)**: 영화나 TV 쇼가 **최초로 공개된 연도**입니다.
- **Y축 (넷플릭스 추가 연도)**: 해당 콘텐츠가 **넷플릭스에 등록된 연도**입니다.
- **값 (각 점)**: **하나의 콘텐츠**를 의미합니다.

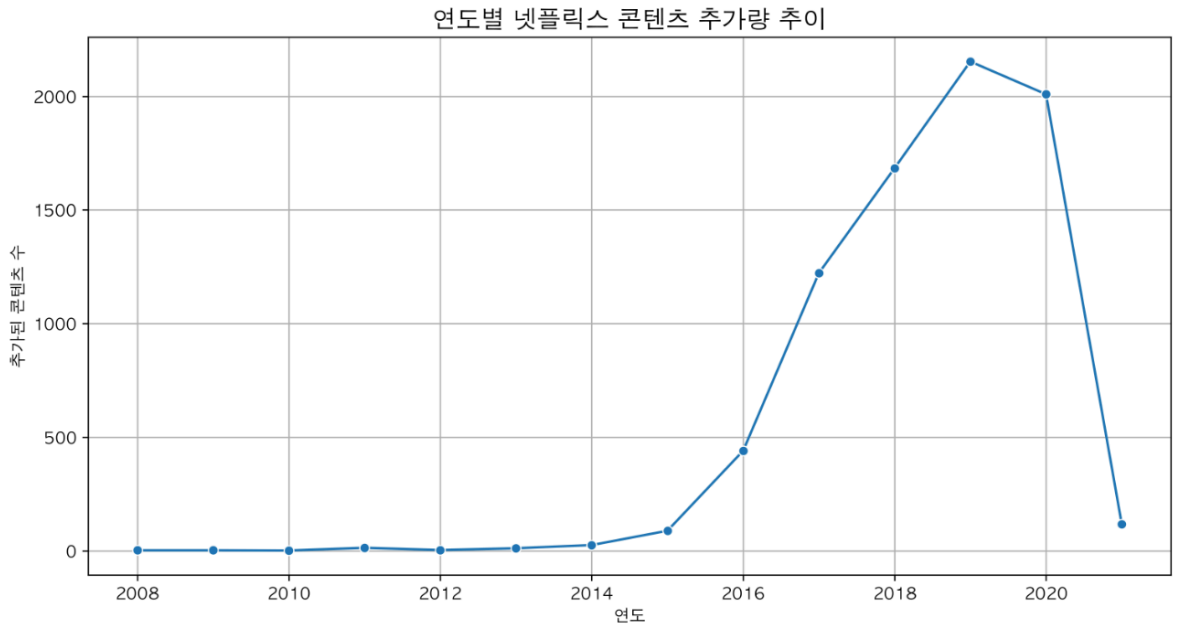
차트 분석 및 추이

- **대각선 분포 (최신 콘텐츠)**: 차트의 오른쪽 상단에는 뚜렷한 대각선 형태의 점들이 밀집해 있습니다. 이 점들은 **제작 연도와 추가 연도가 거의 일치**하는 콘텐츠로, 제작된 지 얼마 안 된 최신 작품이나 '넷플릭스 오리지널'에 해당합니다.

> - **대각선 아래 분포 (구작 콘텐츠)**: 대각선 아래에 넓게 퍼져 있는 점들은 **제작된 지 한참 지난 후에 넷플릭스에 추가된 '구작' 콘텐츠**들입니다. 예를 들어, Y축 2020년 라인을 수평으로 따라가 보면, 2020년에 추가된 콘텐츠 중에는 1960년대, 1980년대 등 매우 오래된 작품들도 포함되어 있음을 알 수 있습니다.
> - **전체적인 통찰**: 이 차트는 넷플릭스가 **두 가지 전략을 동시에 사용**하고 있음을 보여줍니다. 즉, 최신 트렌드를 반영하는 신작들을 빠르게 수급하는 동시에(대각선 분포), 오래된 명작이나 인기 있었던 작품들을 라이브러리에 추가하여(대각선 아래 분포) 콘텐츠의 다양성을 확보하고 있습니다.

```
### 5-3. 시간에 따른 변화 시각화
# 시간의 흐름에 따라 데이터가 어떻게 변하는지 추세를 파악하기 위한 시각화입니다.

# --- 1) 라인 차트 (Line Chart): 시간 흐름에 따른 데이터 변화 ---
# 넷플릭스에 콘텐츠가 추가된 연도별 추이를 라인 차트로 확인합니다.
line_data = df['added_year'].value_counts().sort_index()
plt.figure(figsize=(12, 6))
sns.lineplot(x=line_data.index, y=line_data.values, marker='o')
plt.title('연도별 넷플릭스 콘텐츠 추가량 추이', fontsize=15)
plt.xlabel('연도')
plt.ylabel('추가된 콘텐츠 수')
plt.grid(True)
save_path = os.path.join(output_dir, '5_3_linechart_added_year.png')
plt.savefig(save_path, dpi=300)
plt.close()
print(f"'{save_path}' 파일이 저장되었습니다.")
```



> **차트 분석 및 추이**
>
> - **초기 (2008-2014년)**: 차트의 왼쪽 부분은 거의 0에 가까운 낮은 수치를 보입니다. 이는 넷플릭스가 스트리밍 서비스 초창기에는 콘텐츠 라이브러리를 공격적으로 확장하지 않았음을 보여줍니다.
> - **성장기 (2015-2019년)**: 2015년을 기점으로 선이 가파르게 상승하기 시작하며, 2016년부터 폭발적인 성장세를 보입니다. 이는 넷플릭스가 글로벌 시장으로 확장하고 '넷플릭스 오리지널' 콘텐츠 제작에 막대한 투자를 시작한 시기와 일치합니다. 2019년에 약 2,000개 이상의 콘텐츠를 추가하며 정점을 찍습니다.
> - **최근 (2020-2021년)**: 2020년에는 소폭 감소했으며, **2021년에는 급격히 하락**하는 모습을 보입니다. 이는 넷플릭스가 콘텐츠 추가를 중단한 것이 아니라, **우리가 사용하는 데이터가 2021년 초에 수집되어 해당 연도의 전체 데이터가 포함되지 않았기 때문**입니다. 데이터의 한계를 인지하는 것은 분석에서 매우 중요합니다.

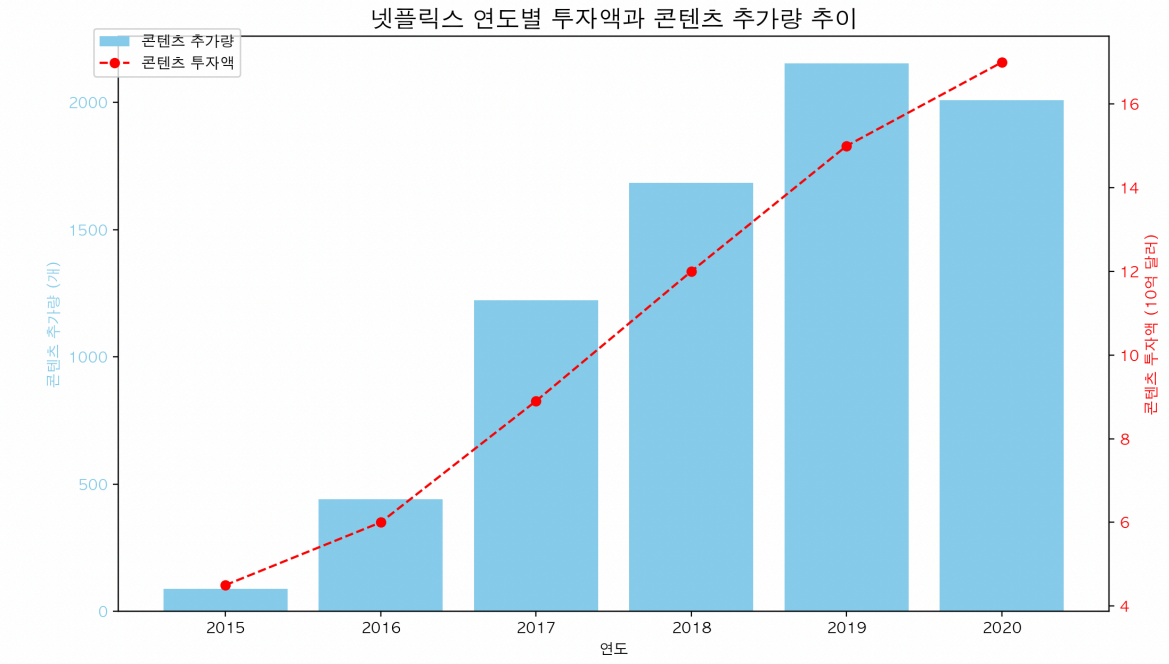
연습문제 2

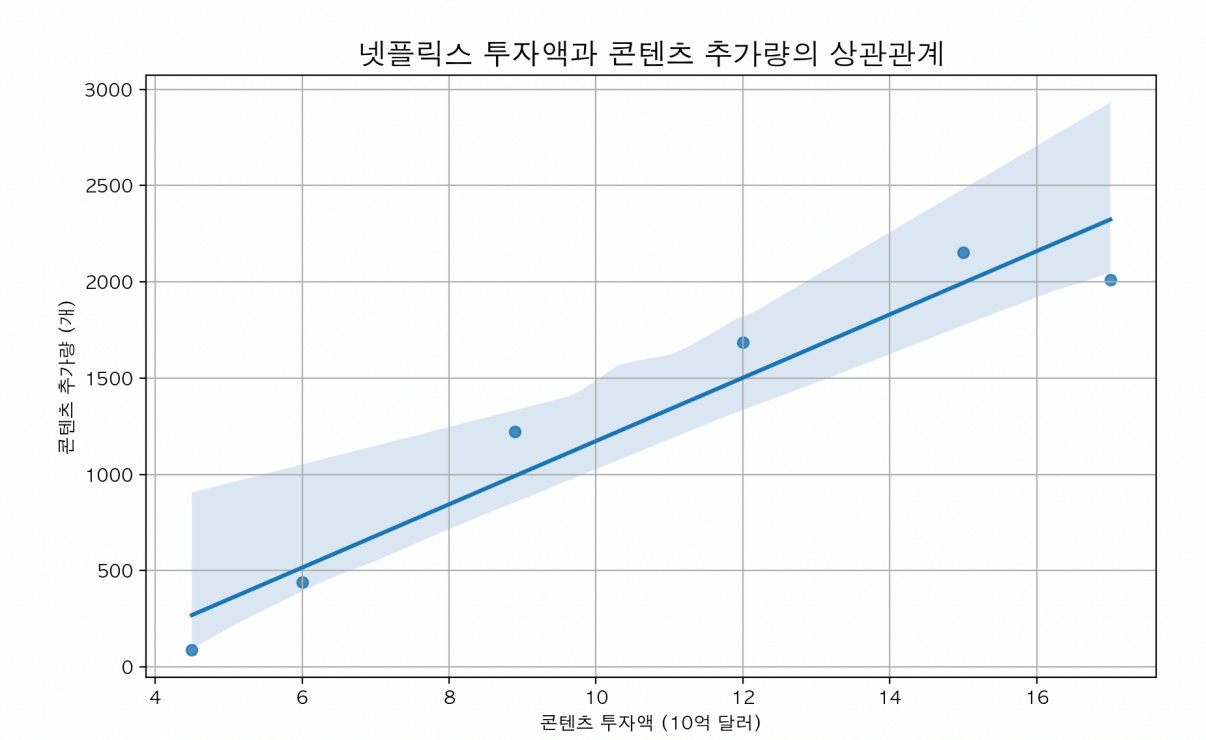
[문제 1] 넷플릭스의 연도별 콘텐츠 추가량과 콘텐츠 투자액(가상 데이터)를 이중 축 차트와 산점도를 사용하여 시각화하세요.

- 투자액 데이터는 임의로 생성합니다, 두 변수 간의 상관관계도 함께 분석하세요. (아래 코드 참고)

```
# 1. 연도별 콘텐츠 추가량 데이터
# (이전 실습에서 전처리가 완료된 df가 있다고 가정)
content_counts = df['added_year'].value_counts().sort_index
()
content_df = content_counts.reset_index()
content_df.columns = ['year', 'added_content']

# 2. 연도별 투자액 가상 데이터 생성
# (실제 데이터가 아닌, 알려진 트렌드를 반영한 예시 데이터입니다)
investment_data = {
    'year': [2015, 2016, 2017, 2018, 2019, 2020],
    'investment_billion_usd': [4.5, 6.0, 8.9, 12.0, 15.0, 1
7.0]
}
investment_df = pd.DataFrame(investment_data)
```





6장: 데이터 분석 심화

지금까지 우리는 데이터의 개별 특징을 탐색(4장)하고 시각화(5장)하는 방법을 배웠습니다. 6장에서는 한 걸음 더 나아가, **데이터의 숨겨진 패턴과 변수들 간의 복잡한 관계**를 파헤치는 심화 분석 기법을 다룹니다. 이를 통해 우리는 더 깊이 있는 인사이트와 질문에 대한 설득력 있는 답변을 찾을 수 있게 됩니다.

6-1. 관계 분석

관계 분석은 **두 개 이상의 변수들이 서로에게 어떤 영향을 미치는지, 혹은 어떤 연관성을 가지는지**를 파악하는 과정입니다. 예를 들어, '특정 장르의 콘텐츠는 특정 시청 등급과 관련이 있을까?' 또는 '국가별로 주로 제작하는 장르에 차이가 있을까?' 와 같은 질문에 답할 수 있습니다. 이러한 범주형 변수 간의 관계를 분석할 때는 주로 분할표(Contingency Table)를 만들어 각 조합의 빈도를 계산하고, 이를 히트맵(Heatmap)으로 시각화하여 패턴을 직관적으로 파악합니다.

6-1장 전체 코드 (`6-1.py`)

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========

# 1-1. 라이브러리 임포트
import pandas as pd
import os
import urllib.request
import matplotlib.pyplot as plt
import seaborn as sns
import platform

# 1-2. 한글 폰트 설정
if platform.system() == 'Darwin':
    plt.rc('font', family='AppleGothic')
elif platform.system() == 'Windows':
    plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)

# 1-3. 차트 저장 폴더 생성
output_dir = 'results'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# 1-4. 데이터 다운로드 및 로드
file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
if not os.path.exists(file_name):
    urllib.request.urlretrieve(url, file_name)
df = pd.read_csv(file_name)

# 1-5. 데이터 전처리 (3장 요약)
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
```

```
df.dropna(subset=['date_added', 'rating'], inplace=True)
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])
df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.month
df['duration_value'] = df['duration'].apply(lambda x: int
(x.split()[0]))
df['duration_unit'] = df['duration'].apply(lambda x: x.spli
t()[1].replace('s',''))

print("데이터 준비 완료. 6-1장 관계 분석을 시작합니다.")
print("=" * 50)



# ============================================================
==========
#                2. 관계 분석 (Relationship Analysis)
# ============================================================
==========

### 6-1.1. 장르와 시청 등급 간 관계 분석
# 특정 장르가 특정 시청 등급과 강한 연관성을 보이는지 확인합니다.
# 예를 들어, '어린이' 장르는 '전체관람가' 등급과 관련이 깊을 것이라
예상할 수 있습니다.

# --- 1) 데이터 준비 ---
# listed_in(장르) 컬럼은 여러 값이 섞여 있으므로, explode를 사용
하여 각 장르를 개별 행으로 분리합니다.
genre_df = df.copy()
genre_df['genre'] = genre_df['listed_in'].str.split(', ')
genre_df = genre_df.explode('genre')

# 너무 많은 조합을 피하기 위해, 가장 빈도가 높은 상위 10개 장르와 상
위 5개 시청 등급만 선택합니다.
top_10_genres = genre_df['genre'].value_counts().head(10).i
ndex
top_5_ratings = genre_df['rating'].value_counts().head(5).i
ndex
```

```
filtered_df = genre_df[
    genre_df['genre'].isin(top_10_genres) &
    genre_df['rating'].isin(top_5_ratings)
]

# --- 2) 분할표(Crosstab) 생성 ---
# 두 변수 간의 빈도를 계산하여 표를 만듭니다.
genre_rating_crosstab = pd.crosstab(filtered_df['genre'], f
iltered_df['rating'])
print("\n### 장르-시청 등급 분할표 ###")
print(genre_rating_crosstab)

# --- 3) 히트맵(Heatmap) 시각화 ---
plt.figure(figsize=(12, 8))
sns.heatmap(genre_rating_crosstab, cmap='YlGnBu', annot=Tru
e, fmt='d', linewidths=.5)
plt.title('주요 장르와 시청 등급 간의 관계', fontsize=16)
plt.xlabel('시청 등급')
plt.ylabel('장르')
save_path = os.path.join(output_dir, '6_1_heatmap_genre_rat
ing.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close()
print(f"\n'{save_path}' 파일이 저장되었습니다.")
# [해석]: 히트맵의 진한 색상을 보면, 'Dramas'와 'International M
ovies'는 'TV-MA'(성인용) 등급과 강한 연관성을 보입니다.
# 반면 'Children & Family Movies'는 'TV-Y', 'TV-G'(전체관람
가)와 높은 빈도를 보일 것임을 예상할 수 있습니다(상위 5개 등급에는
포함되지 않아 표에는 없음).
print("-" * 50)


### 6-1.2. 국가별 작품 제작 분포 분석 🌍
# 국가별로 주로 제작하는 장르에 차이가 있는지 분석합니다.
# 예를 들어, '미국'은 코미디를, '한국'은 드라마를 더 많이 제작할 것
이라 예상해볼 수 있습니다.
```

```
# --- 1) 데이터 준비 ---
# 분석의 편의를 위해, 각 콘텐츠의 첫 번째 국가와 첫 번째 장르를 '주요 국가', '주요 장르'로 간주합니다.
country_df = df.copy()
country_df['main_country'] = country_df['country'].apply(lambda x: x.split(',')[0])
country_df['main_genre'] = country_df['listed_in'].apply(lambda x: x.split(',')[0])

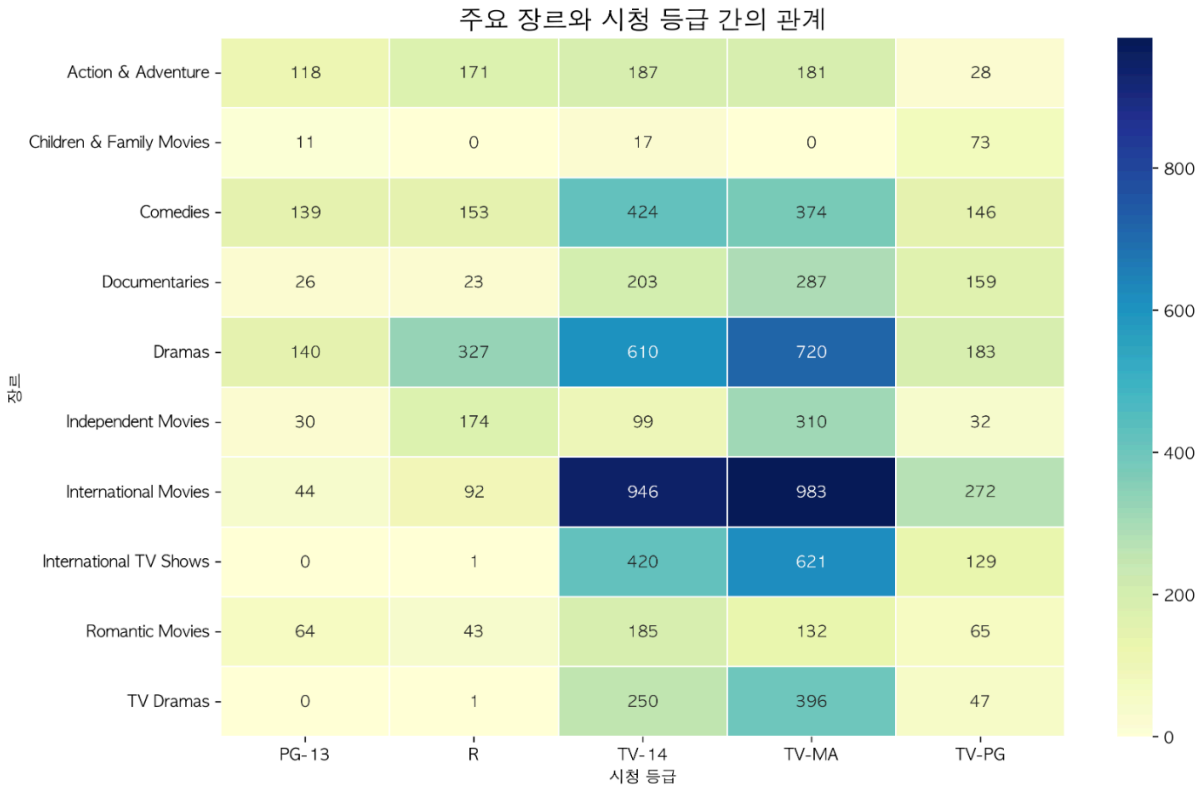
# 'Unknown' 국가와 빈도수 상위 10개 국가, 상위 10개 장르만 선택합니다.
known_countries_df = country_df[country_df['main_country'] != 'Unknown']
top_10_countries = known_countries_df['main_country'].value_counts().head(10).index
top_10_main_genres = known_countries_df['main_genre'].value_counts().head(10).index

filtered_country_df = known_countries_df[
    known_countries_df['main_country'].isin(top_10_countries) &
    known_countries_df['main_genre'].isin(top_10_main_genres)
]

# --- 2) 분할표(Crosstab) 생성 ---
country_genre_crosstab = pd.crosstab(filtered_country_df['main_country'], filtered_country_df['main_genre'])
print("\n### 주요 국가-장르 분할표 ###")
print(country_genre_crosstab)

# --- 3) 히트맵(Heatmap) 시각화 ---
plt.figure(figsize=(15, 10))
sns.heatmap(country_genre_crosstab, cmap='rocket_r', annot=True, fmt='d', linewidths=.5)
plt.title('주요 국가별 선호 장르 분포', fontsize=16)
plt.xlabel('주요 장르')
plt.ylabel('주요 국가')
```

```
save_path = os.path.join(output_dir, '6_1_heatmap_country_genre.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close()
print(f"\n'{save_path}' 파일이 저장되었습니다.")
# [해석]: 히트맵을 통해 국가별 장르 제작의 특성을 파악할 수 있습니다.
# 예를 들어, 미국(United States)은 'Dramas'와 'Comedies'에 강점을 보입니다.
# 인도(India)는 'Dramas'와 'International Movies'에 집중하는 경향을, 일본(Japan)은 'Anime Series'에 독보적인 강점을 보입니다.
```



📌

1. 주요 장르와 시청 등급 간의 관계

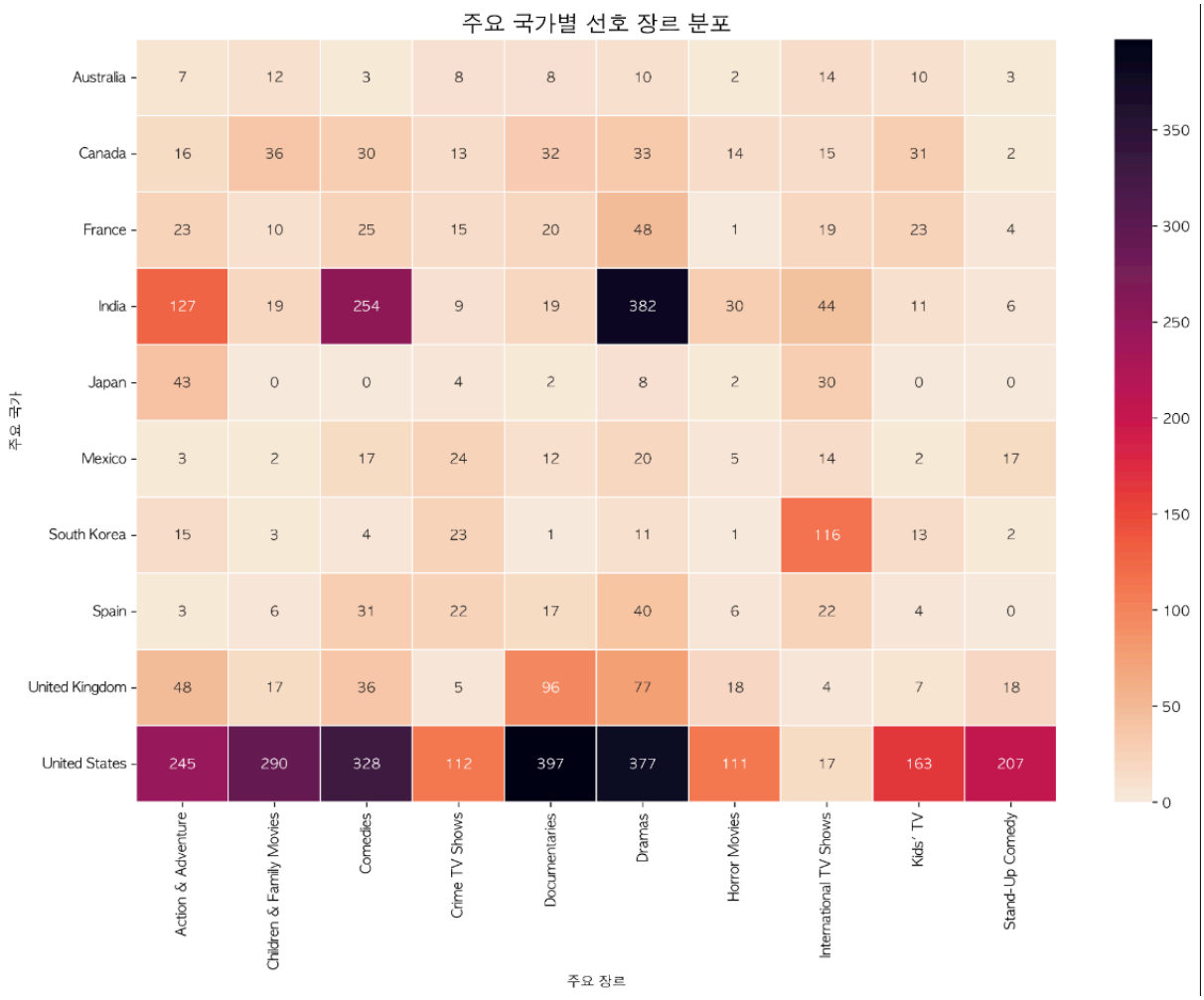
이 차트는 **어떤 장르가 어떤 시청 등급과 강한 연관성을 보이는지**를 보여줍니다. 각 셀의 숫자는 특정 장르와 특정 시청 등급의 조합을 가진 콘텐츠의 개수이며, 색이 진할수록 그 수가 많다는 의미입니다.

분석 및 인사이트

- 성인/청소년 중심의 콘텐츠: `TV-MA`(성인용)와 `TV-14`(14세 이상) 컬럼의 색상이 전반적으로 가장 진합니다. 이는 넷플릭스의 콘텐츠 라이브러리가 **성인과 청소년 시청자층을 핵심 타겟**으로 하고 있음을 명확하게 보여줍니다.

- 강력한 장르-등급 조합: `International Movies`**와** `Dramas`: 이 두 장르는 `TV-MA`와 `TV-14` 등급에서 압도적으로 높은 수치(600~900대)를 보입니다. 즉, 넷플릭스에 있는 해외 영화와 드라마는 대부분 성인이나 청소년을 위한 콘텐츠임을 알 수 있습니다. `Comedies`**와** `Documentaries`: 이 장르들 역시 `TV-MA`와 `TV-14` 등급에 집중되어, 가벼운 코미디나 다큐멘터리조차도 성인 취향의 콘텐츠가 많다는 것을 시사합니다.

- 명확한 타겟팅: `Children & Family Movies` 장르는 예상대로 `TV-PG`(부모 동반 시청) 등급에서 가장 높은 수치(73)를 보이며, `R`이나 `TV-MA` 등급은 0으로 나타납니다. 이는 특정 장르에 대한 명확한 시청 등급 타겟팅이 이루어지고 있음을 보여줍니다.



📌 2. 주요 국가별 선호 장르 분포

이 차트는 **주요 국가별로 어떤 장르의 콘텐츠를 주로 제작하는지 그 특성과 강점**을 시각적으로 비교합니다. 각 셀의 숫자는 특정 국가가 특정 장르로 제작한 콘텐츠의 수입니다.

- **미국의 압도적 영향력**: `United States` 행은 대부분의 장르에 걸쳐 색상이 진하게 나타납니다. 이는 넷플릭스 라이브러리 내에서 미국이 **양적으로 가장 많은 콘텐츠를, 그리고 장르적으로도 가장 다양한 콘텐츠를 공급**하고 있음을 보여줍니다.
- **국가별 장르 특화** 🌍:
  - **인도 (`India`)**: `Dramas`(382개)와 `Comedies`(254개)에서 매우 강한 집중도를 보입니다. 이는 '발리우드'로 대표되는 인도 영화 산업의 특징이 넷플릭스 콘텐츠에도 그대로 반영된 결과입니다.
  - **대한민국 (`South Korea`)**: 다른 모든 장르를 압도하는 `International TV Shows`(116개)가 독보적으로 높은 수치를 기록합니다. 이는 전 세계적인

K-드라마의 위상을 명확히 보여주는 데이터입니다.

    - **일본 (`Japan`)**: `Action & Adventure`(43개)와 `International TV Shows`(30개)에서 강세를 보이며, 이는 주로 '애니메이션' 장르의 영향으로 해석할 수 있습니다.
    - **영국 (`United Kingdom`)**: `Documentaries`(96개)와 `Dramas`(77개) 제작에 강점을 보여, BBC 다큐멘터리나 영국 시대극과 같은 전통적인 강세 장르가 반영된 것을 알 수 있습니다.
- **종합**: 이 히트맵은 각 국가의 문화적 콘텐츠 생산 강점이 넷플릭스라는 글로벌 플랫폼에 어떻게 분포되어 있는지를 보여주는 '문화 지도'와 같은 역할을 합니다.

6-2. 네트워크 분석

네트워크 분석은 **개체(Node)들과 그들 사이의 관계(Edge)로 이루어진 '네트워크'의 구조와 특성을 분석**하는 기법입니다. 소셜 미디어의 친구 관계, 논문 인용 관계, 그리고 영화 속 감독-배우 협업 관계 등이 모두 네트워크 분석의 대상이 될 수 있습니다.

이를 통해 우리는 전체 구조 속에서 **가장 영향력 있는 중심 인물이 누구인지(중심성 분석)**, 어떤 그룹들이 긴밀하게 협력하는지(커뮤니티 탐지) 등을 파악할 수 있습니다. 이번 실습에서는 파이썬의 대표적인 네트워크 분석 라이브러리인 `NetworkX`를 사용하여, 어떤 감독과 배우가 협업 네트워크의 중심에 있는지 알아보겠습니다.

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========

# 1-1. 라이브러리 임포트
import pandas as pd
import os
import urllib.request
import matplotlib.pyplot as plt
import seaborn as sns
import platform
import networkx as nx # 네트워크 분석 라이브러리
```

```
# 1-2. 한글 및 특수문자 폰트 설정
if platform.system() == 'Darwin': # macOS의 경우
    plt.rc('font', family='D2Coding')
elif platform.system() == 'Windows': # Windows의 경우
    plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)

# 1-3. 차트 저장 폴더 생성
output_dir = 'results'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# 1-4. 데이터 다운로드 및 로드
file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
if not os.path.exists(file_name):
    urllib.request.urlretrieve(url, file_name)
df = pd.read_csv(file_name)

# 1-5. 데이터 전처리 (3장 요약)
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
df.dropna(subset=['date_added', 'rating'], inplace=True)
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])
df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.month
df['duration_value'] = df['duration'].apply(lambda x: int
(x.split()[0]))
df['duration_unit'] = df['duration'].apply(lambda x: x.spli
t()[1].replace('s',''))

print("데이터 준비 완료. 6-1장 관계 분석을 시작합니다.")
print("=" * 50)
```

```
# ============================================================
#            2. 감독·배우 네트워크 분석 (NetworkX 활용)
# ============================================================
# 전체 데이터를 사용하면 네트워크가 너무 복잡해지므로, 상위 감독들을 중심으로 분석 범위를 좁힙니다.

# --- 1) 분석 데이터 준비 ---
# 'Unknown' 감독 제외 및 주요 감독(첫 번째 이름) 컬럼 생성
df_net = df[df['director'] != 'Unknown'].copy()
df_net['main_director'] = df_net['director'].apply(lambda x: x.split(',')[0])

# 가장 많은 작품을 제작한 상위 30명의 감독을 선택
top_directors = df_net['main_director'].value_counts().head(30).index
subset_df = df_net[df_net['main_director'].isin(top_directors)]

# --- 2) 엣지 리스트(Edge List) 생성 ---
# 네트워크의 관계(Edge)를 정의합니다. 여기서는 (감독, 배우) 쌍으로 관계를 정의합니다.
edge_list = []
for index, row in subset_df.iterrows():
    director = row['main_director']
    cast = row['cast'].split(', ')
    # 각 감독에 대해 출연한 모든 배우와의 관계를 튜플로 만들어 리스트에 추가
    for actor in cast:
        if actor != 'Unknown':
            edge_list.append((director, actor))

print(f"상위 30명 감독과 협업한 배우 관계(엣지) 수: {len(edge_list)}개")
```

```
# --- 3) NetworkX 그래프 생성 및 시각화 ---
G = nx.Graph()
G.add_edges_from(edge_list)

# 노드(감독/배우)의 색상 및 크기 설정
node_colors = []
node_sizes = []
for node in G.nodes():
    if node in top_directors:
        node_colors.append('tomato') # 감독은 빨간색
        node_sizes.append(150)
    else:
        node_colors.append('skyblue') # 배우는 하늘색
        node_sizes.append(30)

# 시각화
plt.figure(figsize=(20, 20))
# Kamada-Kawai 레이아웃은 노드 간의 거리를 최적화하여 관계를 보기 좋게 배치합니다.
pos = nx.kamada_kawai_layout(G)

# 네트워크 그리기
nx.draw_networkx_nodes(G, pos, node_color=node_colors, node_size=node_sizes)
nx.draw_networkx_edges(G, pos, alpha=0.2, edge_color='gray')

# 감독 이름만 라벨로 표시 (배우까지 표시하면 너무 복잡함)
director_labels = {node: node for node in G.nodes() if node in top_directors}
nx.draw_networkx_labels(G, pos, labels=director_labels, font_family='D2Coding', font_size=12)

plt.title('주요 감독-배우 협업 네트워크', fontsize=20)
plt.axis('off') # 축 숨기기
save_path = os.path.join(output_dir, '6_2_network_director_actor.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
```

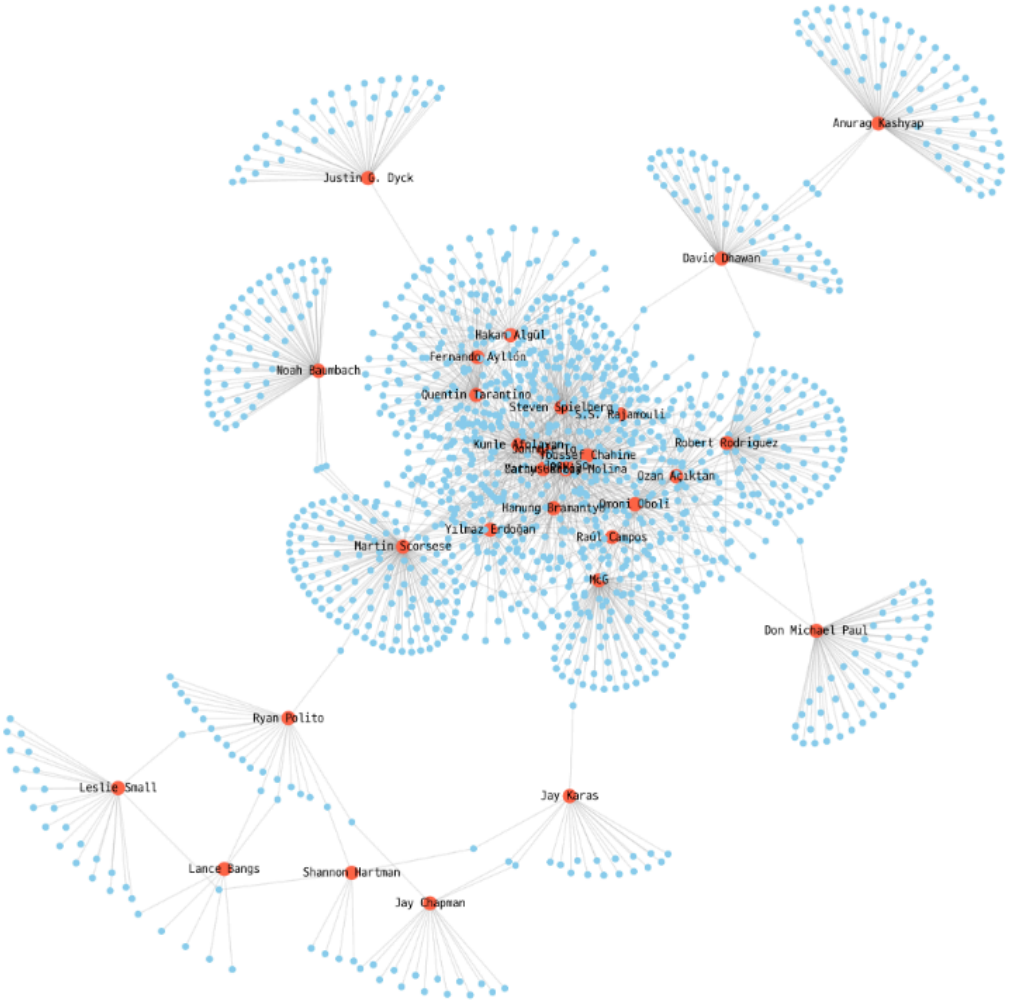
```
plt.close()
print(f"\n'{save_path}' 파일이 저장되었습니다.")


# --- 4) 중심성(Centrality) 분석 ---
# 네트워크에서 가장 영향력 있는 노드를 찾습니다. '연결 중심성(Degree Centrality)'은
# 한 노드가 다른 몇 개의 노드와 직접적으로 연결되어 있는지를 나타내는 가장 기본적인 척도입니다.

# 연결 중심성 계산
degree_centrality = nx.degree_centrality(G)
# 중심성이 높은 상위 15개 노드(감독 또는 배우)를 정렬하여 출력
sorted_centrality = sorted(degree_centrality.items(), key=lambda item: item[1], reverse=True)

print("\n### 네트워크 연결 중심성 상위 15명 (감독/배우) ###")
for node, centrality in sorted_centrality[:15]:
    print(f"{node}: {centrality:.4f}")
print("-" * 50)
```

주요 감독-배우 협업 네트워크



1. 네트워크 그래프(이미지) 분석

이 네트워크 그래프는 **상위 30명의 감독(빨간 점)과 그들과 협업한 배우들(파란 점) 사이의 관계**를 시각화한 것입니다.

- 거대한 중앙 클러스터 (The Mainstream Hub):
  차트 중앙에는 Martin Scorsese, Steven Spielberg, Quentin Tarantino 등 여러 감독 노드들이 서로 밀집해 있고, 수많은 배우 노드(파란 점)를 공유하며 거대한 덩어리를 형성하고 있습니다. 이는 이 감독들이 서로 비슷한 배우 풀 내에서 작업하며, 할리우드와 같이 서로 긴밀하게 연결된 주류 제작 생태계를 구성하고 있음을 시각적으로 보여줍니다.
- 독립적인 외부 클러스터 (The Independent Hubs):

중앙 클러스터 바깥쪽에는 Anurag Kashyap, David Dhawan, Don Michael Paul 등 몇몇 감독들이 자신만의 배우 그룹과 함께 독립적인 '별 모양'의 클러스터를 형성하고 있습니다. 이는 이 감독들이 자신만의 사단(페르소나)을 구축하여 반복적으로 협업하거나, 특정 국가(예: 인도)나 장르 내에서 활동하여 중앙의 주류 생태계와는 배우 교류가 적다는 것을 의미합니다.

- **종합**: 이 그래프는 넷플릭스에 등재된 작품들을 기준으로 볼 때, 여러 감독이 배우를 공유하는 **거대 협업망**과 특정 감독을 중심으로 뭉치는 **독립적 협업망**이 공존하는 모습을 명확하게 보여줍니다.

---

2. 중심성 분석 (실행 로그) 분석

실행 로그에 나타난 '연결 중심성(Degree Centrality)'은 한 명의 인물(노드)이 네트워크 내에서 얼마나 많은 다른 인물과 직접적으로 연결되어 있는지를 나타내는 척도입니다. 즉, **가장 많은 파트너와 협업한 인물**을 찾는 것입니다.

- **핵심 발견**: 중심성 상위 15명 목록에 **모두 감독의 이름만** 올라와 있습니다. 배우의 이름은 보이지 않습니다.
- 이유 분석 (매우 중요):
  이것은 배우가 영향력이 없다는 뜻이 아니라, 우리가 설정한 분석 방식 때문에 나타나는 자연스러운 결과입니다. 이번 분석에서 '연결'은 (감독-배우) 관계로 정의되었습니다. 따라서 한 감독은 자신의 영화에 출연한 배우 수만큼(예: 50명)의 연결을 갖게 됩니다. 반면, 한 배우는 여러 감독의 영화에 출연해야만 연결 수가 늘어납니다.
  결과적으로, 대규모 캐스팅을 한 감독 한 명의 연결 수가 수십 편의 영화에 출연한 배우 한 명의 연결 수보다 훨씬 많아지기 때문에 감독들의 중심성 점수가 높게 나올 수밖에 없습니다.
- **결론**: 이 중심성 순위는 "네트워크에서 가장 영향력 있는 인물"이라기보다는, "이번 분석 대상인 상위 30명의 감독 중, 가장 다양한 배우들과 협업한 감독 순위"라고 해석하는 것이 가장 정확합니다. **Martin Scorsese** 감독이 넷플릭스 데이터 내에서 가장 폭넓은 배우 캐스팅 풀을 가졌음을 의미합니다.

6-3. 텍스트 마이닝

지금까지 우리는 숫자나 범주처럼 정형화된 데이터를 분석했습니다. 하지만 데이터의 상당수는 영화 줄거리(`description`)처럼 비정형적인 **텍스트 데이터**로 이루어져 있습니다. 텍스트 마이닝(Text Mining)은 이러한 텍스트 데이터에서 의미 있는 정보와 패턴을 추출하는 기술입니다. 텍스트 마이닝을 통해 우리는 "넷플릭스 콘텐츠 설명에 공통으로 나타나는 핵심

키워드는 무엇일까?" 또는 "특정 장르의 영화는 어떤 주제를 주로 다룰까?"와 같은 질문에 답할 수 있습니다.

이번 실습에서는 텍스트 데이터 시각화의 가장 대표적인 기법인 워드 클라우드(Word Cloud)를 만들어 보겠습니다. 워드 클라우드는 텍스트에서 단어의 등장 빈도를 분석하여, 빈도가 높을수록 단어를 더 크게 표현하는 시각화 기법입니다. 이를 통해 텍스트 전체의 핵심 주제를 직관적으로 파악할 수 있습니다.

> ※ 라이브러리 설치
>
> 이 실습을 진행하기 전에, 터미널 또는 Anaconda Prompt에서 `wordcloud` 라이브러리를 설치해야 합니다.
>
> `pip install wordcloud`

```
# ============================================================
#                       1. 데이터 준비 (Setup)
# ============================================================

# 1-1. 라이브러리 임포트
import pandas as pd
import os
import urllib.request
import matplotlib.pyplot as plt
from wordcloud import WordCloud, STOPWORDS # 워드 클라우드 라이브러리

# 1-2. 한글 폰트 설정
plt.rc('font', family='D2Coding')
plt.rc('axes', unicode_minus=False)


# 1-3. 차트 저장 폴더 생성
output_dir = 'results'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
```

```
# 1-4. 데이터 다운로드 및 로드 (이전과 동일)
file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
if not os.path.exists(file_name):
    urllib.request.urlretrieve(url, file_name)
df = pd.read_csv(file_name)

# 1-5. 데이터 전처리 (3장 요약)
df['director'] = df['director'].fillna('Unknown')
df['cast'] = df['cast'].fillna('Unknown')
df['country'] = df['country'].fillna('Unknown')
df.dropna(subset=['date_added', 'rating'], inplace=True)
df['date_added'] = df['date_added'].str.strip()
df['date_added'] = pd.to_datetime(df['date_added'])
df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.month
df['duration_value'] = df['duration'].apply(lambda x: int
(x.split()[0]))
df['duration_unit'] = df['duration'].apply(lambda x: x.spli
t()[1].replace('s',''))

print("데이터 준비 완료. 6-1장 관계 분석을 시작합니다.")
print("=" * 50)


# ============================================================
==========
#        2. 영화 설명(Description) 키워드 분석 및 워드 클라우드
# ============================================================
==========

### 6-3.1. 전체 콘텐츠 대상 워드 클라우드 생성

# --- 1) 텍스트 데이터 통합 ---
# 'description' 컬럼의 모든 문장을 하나의 거대한 텍스트 덩어리로
합칩니다.
```

```
text = " ".join(desc for desc in df['description'])
print(f"총 {len(text)} 자의 텍스트 데이터가 준비되었습니다.")

# --- 2) 불용어(Stopwords) 설정 ---
# 분석에 의미 없는 단어(a, the, is 등)를 제거하기 위해 불용어 사전을 설정합니다.
# 기본 제공되는 불용어에 'show', 'movie' 등 분석에 불필요한 단어를 추가합니다.
stopwords = set(STOPWORDS)
stopwords.update(['show', 'movie', 'series', 'film', 'new', 'one', 'two'])

# --- 3) 워드 클라우드 생성 ---
# WordCloud 객체를 생성하고, 통합된 텍스트를 입력하여 단어 빈도를 계산합니다.
wordcloud = WordCloud(
    stopwords=stopwords,
    background_color='black', # 배경색
    max_words=100,            # 최대 표시 단어 수
    width=800,                # 이미지 너비
    height=400                # 이미지 높이
).generate(text)

# --- 4) 시각화 및 저장 ---
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.title('넷플릭스 콘텐츠 설명 키워드', fontsize=16)
plt.axis('off') # 축 숨기기
save_path = os.path.join(output_dir, '6_3_wordcloud_all.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close()
print(f"\n'{save_path}' 파일이 저장되었습니다.")
# [해석]: 'life', 'young', 'family', 'world', 'love', 'friends' 등의 단어가 크게 나타납니다.
# 이는 넷플릭스 콘텐츠가 주로 '삶', '가족', '사랑', '우정'과 같은 인간적인 주제를 다루고 있음을 보여줍니다.
print("-" * 50)
```

```
### 6-3.2. [심화] 특정 장르별 워드 클라우드 비교
# 장르에 따라 핵심 키워드가 어떻게 다른지 비교해 봅시다.

def generate_genre_wordcloud(genre_name, file_name_suffix):
    """특정 장르의 워드 클라우드를 생성하고 저장하는 함수"""
    print(f"'{genre_name}' 장르의 워드 클라우드를 생성합니
다...")
    # 해당 장르가 포함된 설명만 필터링
    genre_text = " ".join(desc for desc in df[df['listed_i
n'].str.contains(genre_name)]['description'])

    wordcloud = WordCloud(
        stopwords=stopwords,
        background_color='white',
        max_words=100,
        width=800,
        height=400,
        colormap='viridis' # 다른 색상 맵 적용
    ).generate(genre_text)

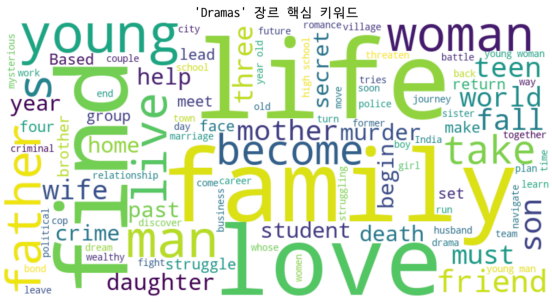
    plt.figure(figsize=(12, 6))
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.title(f"'{genre_name}' 장르 핵심 키워드", fontsize=1
6)
    plt.axis('off')
    save_path = os.path.join(output_dir, f'6_3_wordcloud_{f
ile_name_suffix}.png')
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close()
    print(f"'{save_path}' 파일이 저장되었습니다.\n")

# 'Dramas'와 'Comedies' 장르에 대해 워드 클라우드 생성
generate_genre_wordcloud('Dramas', 'dramas')
generate_genre_wordcloud('Comedies', 'comedies')
```

넷플릭스 콘텐츠 설명 키워드



'Dramas' 장르 핵심 키워드



'Comedies' 장르 핵심 키워드





워드 클라우드 종합 분석

1. 전체 넷플릭스 콘텐츠 키워드 (상단)

이 워드 클라우드는 넷플릭스의 모든 콘텐츠 설명을 종합하여 핵심 키워드를 보여줍니다.

- **가장 큰 단어**: `life`, `love`, `family`, `world`, `woman`, `man` 등이 가장 크게 나타납니다.
- **분석**: 넷플릭스 콘텐츠의 대주제는 '삶과 관계'임을 알 수 있습니다. 장르를 불문하고 '인생', '사랑', '가족', '세상' 등 보편적이고 인간 중심적인 이야기가 주를 이룹니다. `young`, `friends`, `help` 와 같은 단어들은 성장과 관계에 대한 이야기가 많다는 것을 시사합니다.

2. 'Dramas' 장르 핵심 키워드 (하단 좌측)

드라마 장르에만 초점을 맞춘 이 워드 클라우드는 전체 클라우드보다 훨씬 더 구체적이고 강렬한 주제를 보여줍니다.

- **가장 큰 단어**: 전체 클라우드와 마찬가지로 `life`, `family`, `love`가 크지만, 그에 못지않게 `young`, `woman`, `man`*이 두드러집니다.
- **주목할 만한 단어**: `secret`, `past`, `murder`, `crime`, `father`, `mother` 와 같은 단어들이 새롭게 중요하게 등장합니다.
- **분석**: 드라마 장르는 보편적인 삶의 주제를 다루면서도, **'과거의 비밀', '범죄와 살인', '깊은 가족 갈등'** 등 훨씬 더 무겁고 갈등 중심적인 서사를 다루는 경향이 짙습니다. `father`, `mother`의 등장은 가족 내의 심층적인 관계나 갈등이 중요한 요소임을 보여줍니다.

---

3. 'Comedies' 장르 핵심 키워드 (하단 우측)

코미디 장르의 키워드는 드라마와 뚜렷한 대조를 보이며, 더 가볍고 사회적인 관계에 집중합니다.

- **가장 큰 단어**: `life`, `friends`, `love`, `family`*가 중심을 이룹니다.
- **주목할 만한 단어**: `friends`*가 드라마 장르보다 훨씬 더 크게 강조되며, `school`, `high`, `team`, `save` 와 같은 단어들이 눈에 띕니다.
- **분석**: 코미디 장르는 심각한 범죄나 과거의 비밀보다는 **'친구 관계', '학창 시절', '사랑'** 등 더 밝고 일상적인 관계에서 벌어지는 이야기에 초점을 맞춥니다. `team`, `save` 와 같은 단어는 함께 어려움을 극복하는 협력적인 서사가 많음을 시사하며, 전반적으로 드라마보다 긍정적이고 가벼운 분위기를 가집니다.

장르별 비교 및 핵심 인사이트

- **공통점**: 세 클라우드 모두 '삶(life)', '사랑(love)', '가족(family)'이라는 인간의 보편적인 주제를 공유합니다. 이는 대부분의 이야기가 결국 사람과 관계에 대한 것임을 보여줍니다.
- **차이점**: **갈등의 종류**에서 장르적 특성이 명확히 드러납니다.
  - **드라마**는 `crime`, `murder`, `secret` 등 무거운 사회적, 개인적 갈등을 다룹니다.
  - **코미디**는 `friends`, `school` 등 사회적, 관계적 상황에서 발생하는 가벼운 갈등을 다룹니다.

결론적으로, 이 워드 클라우드 비교 분석을 통해 우리는 넷플릭스가 보편적인 주제를 바탕으로 하되, 장르에 따라 갈등의 성격과 이야기의 톤을 명확히 구분하여

다양한 시청자층을 공략하고 있음을 알 수 있습니다.

6-4. 추천 시스템 기초: 콘텐츠 기반 필터링

- 추천 시스템(Recommender System)은 사용자의 과거 행동이나 콘텐츠의 특징을 분석하여, 사용자가 좋아할 만한 항목을 예측하고 추천하는 기술입니다. 넷플릭스가 "회원님을 위한 추천 콘텐츠"를 보여주거나, 유튜브가 다음에 볼 동영상을 제안하는 것이 모두 추천 시스템의 예시입니다.
- 추천 시스템에는 여러 방식이 있지만, 우리는 그중 가장 직관적인 콘텐츠 기반 필터링(Content-Based Filtering)을 배워보겠습니다.
- **콘텐츠 기반 필터링**이란, **사용자가 과거에 좋아했던 항목과 '내용(content)이 유사한' 다른 항목을 추천**하는 방식입니다. 예를 들어, 사용자가 '아이언맨'을 재미있게 봤다면, '아이언맨'과 비슷한 특징(장르: 액션/SF, 출연 배우: 로버트 다우니 주니어, 줄거리 키워드: 영웅, 기술)을 가진 '어벤져스'를 추천해 주는 것입니다.
- 이 '유사도'를 계산하기 위해, 우리는 텍스트 데이터를 컴퓨터가 이해할 수 있는 숫자 벡터(vector)로 변환해야 합니다. 이번 실습에서는 **TF-IDF(Term Frequency-Inverse Document Frequency)** 라는 기법을 사용하여 영화 설명(`description`)을 벡터로 만들고, 코사인 유사도(Cosine Similarity)를 통해 영화 간의 유사도를 계산하여 추천 시스템을 구현해 보겠습니다.

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import os
import urllib.request
import matplotlib.pyplot as plt
import seaborn as sns

plt.rc('font', family='D2Coding')
```

```
plt.rc('axes', unicode_minus=False)
output_dir = 'results'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
if not os.path.exists(file_name):
    urllib.request.urlretrieve(url, file_name)
df = pd.read_csv(file_name)

df.drop_duplicates(subset='title', keep='first', inplace=True)
features = ['listed_in', 'director', 'cast', 'description']
for feature in features:
    df[feature] = df[feature].fillna('')

print("데이터 준비 완료. 개선된 추천 시스템 구현을 시작합니다.")
print("=" * 50)

# ============================================================
#       2. [개선] 콘텐츠 기반 필터링 (다중 피처 활용)
# ============================================================
def create_soup(x):
    director = str(x['director']).replace(' ', '').lower()
    cast = str(x['cast']).replace(' ', '').lower()
    listed_in = str(x['listed_in']).replace(' ', '').lower()
    description = str(x['description'])
    return listed_in + ' ' + director + ' ' + cast + ' ' + description

df['soup'] = df.apply(create_soup, axis=1)

tfidf = TfidfVectorizer(stop_words='english')
```

```
tfidf_matrix = tfidf.fit_transform(df['soup'])
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
print("개선된 TF-IDF 및 코사인 유사도 행렬 생성 완료.")

indices = pd.Series(df.index, index=df['title']).drop_duplicates()

def get_recommendations_with_scores(title, cosine_sim=cosine_sim):
    if title not in indices:
        return f"Error: '{title}' not found in the dataset."
    idx = indices[title]
    sim_scores = list(enumerate(cosine_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:11]
    movie_indices = [i[0] for i in sim_scores]
    scores = [i[1] for i in sim_scores]
    result_df = pd.DataFrame({'title': df['title'].iloc[movie_indices], 'score': scores})
    return result_df

target_movie = 'The Matrix'
recommendations_df = get_recommendations_with_scores(target_movie)

print(f"\n### '{target_movie}'와 유사한 콘텐츠 추천 목록 ###")
print(recommendations_df)

# [수정] FutureWarning 해결을 위해 hue, legend 옵션 추가
plt.figure(figsize=(10, 6))
sns.barplot(x='score', y='title', data=recommendations_df, palette='viridis', hue='title', legend=False)
plt.title(f"'{target_movie}' 추천 결과 및 유사도 점수", fontsize=15)
plt.xlabel("코사인 유사도 점수")
plt.ylabel("추천 콘텐츠")
```

```
save_path = os.path.join(output_dir, '6_4_recommendation_The_Matrix.png')
plt.savefig(save_path, dpi=300, bbox_inches='tight')
plt.close()
print(f"\n'{save_path}' 파일이 저장되었습니다.")
```

6-5. 협업 필터링 심화: 아이템 기반 필터링

이전 실습에서 우리는 가상의 데이터를 통해 협업 필터링의 강력한 성능을 확인했습니다. 이번 장에서는 그중에서도 우리가 구현했던 아이템 기반 협업 필터링(Item-Based Collaborative Filtering)이 어떤 원리로 동작하는지 그 이론적 배경을 깊이 있게 알아보겠습니다.

핵심 아이디어: "이 영화를 좋아한 사람들은, 다른 어떤 영화를 좋아했을까?"

콘텐츠 기반 필터링이 "이 영화와 **내용이 비슷한** 영화는 무엇일까?"라고 질문했다면, 아이템 기반 협업 필터링은 "이 영화와 비슷하게 평가받은 영화는 무엇일까?"라고 질문합니다. 여기서 '비슷하게 평가받았다'는 것은 **수많은 사용자의 평점 패턴**을 기반으로 합니다.

가장 쉬운 비유는 **마트의 장바구니 분석**입니다.

> 수많은 고객들의 구매 데이터를 분석했더니, '우유를 구매한 사람들이 계란도 함께 구매하는' 강력한 패턴을 발견했습니다. 이 패턴을 통해 우리는 '우유'와 '계란'이 내용물은 전혀 다르지만, 소비 패턴상으로는 매우 유사한 상품이라고 정의할 수 있습니다.
>
> 이제, 새로운 고객이 우유를 카트에 담으면, 우리는 그에게 계란을 추천해 줄 수 있습니다.

아이템 기반 협업 필터링도 이와 동일합니다. '우유'는 영화 <매트릭스>이고, '계란'은 영화 <인셉션>입니다. 수많은 사용자들이 <매트릭스>와 <인셉션>에 모두 높은 평점을 줬다면, 두 영화는 장르나 배우가 달라도 **취향 패턴상으로는 매우 유사한 아이템**으로 간주됩니다.

동작 원리 3단계

아이템 기반 협업 필터링은 다음과 같은 3단계를 거쳐 추천을 생성합니다.

- 1단계: 사용자-아이템 행렬 생성 (User-Item Matrix)

모든 사용자의 평점 데이터를 기반으로, 행(row)은 사용자, 열(column)은 아이템(영화), 그리고 값(value)은 평점인 거대한 행렬을 만듭니다. 대부분의 사용자는 전체 영화 중 극히 일부만 평가했으므로, 이 행렬의 대부분은 비어있는 상태(NaN)가 됩니다.

- 2단계: 아이템 간 유사도 계산 (Item-Item Similarity)

사용자-아이템 행렬의 열(column)들을 서로 비교하여 모든 영화 쌍(pair) 간의 유사도를 계산합니다. 이때 주로 피어슨 상관계수(Pearson Correlation)가 사용됩니다.

- **피어슨 상관계수**: 두 영화에 대해 공통으로 평점을 매긴 모든 사용자의 점수 패턴이 얼마나 일치하는지를 측정합니다.
  - **+1에 가까울수록**: 한 영화를 높게 평가한 사용자들이 다른 영화도 높게 평가하는 경향 (강한 양의 상관관계)
  - **1에 가까울수록**: 한 영화를 높게 평가한 사용자들이 다른 영화는 낮게 평가하는 경향 (강한 음의 상관관계)
  - **0에 가까울수록**: 두 영화의 평점 간에 뚜렷한 관계가 없음

이 계산이 끝나면, 어떤 영화 두 개를 고르더라도 그 사이의 유사도 점수를 바로 알 수 있는 **아이템 유사도 행렬**이 완성됩니다.

- 3단계: 추천 생성 (Generate Recommendations)

이제 특정 사용자에게 영화를 추천하는 것은 간단합니다.

1. 사용자가 최근에 재미있게 본 영화(예: <매트릭스>)를 기준으로 삼습니다.
2. 아이템 유사도 행렬에서 <매트릭스>와 유사도 점수가 가장 높은 영화들을 순서대로 찾습니다. (예: 1위 <인셉션>, 2위 <V for Vendetta>...)
3. 이 목록에서 사용자가 이미 본 영화는 제외합니다.
4. 나머지 목록의 상위 N개 영화를 최종 추천 결과로 제시합니다.

장점과 단점

- **장점**:
  - **안정적인 추천**: 사람의 취향은 변할 수 있지만, 아이템 간의 관계(예: <매트릭스>와 <인셉션>의 유사성)는 잘 변하지 않아 안정적입니다.
  - **새로운 발견(Serendipity)**: 콘텐츠 내용과 상관없이 다른 사람들의 숨겨진 취향 패턴에 기반하므로, 생각지 못했던 새로운 장르의 영화를 추천해 줄 수 있습니다.
- **단점**:
  - **콜드 스타트(Cold Start) 문제**: 아무도 평점을 매기지 않은 **신규 아이템**은 다른 아이템과의 유사도를 계산할 수 없어 추천이 불가능합니다.

- **데이터 부족 문제**: 사용자와 평점 데이터가 적으면 의미 있는 유사도 패턴을 찾기 어렵습니다.

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========

import pandas as pd

# 1-1. [중요] 가상의 사용자 평점(User-Rating) 데이터 생성
# 실제 데이터셋에는 이 정보가 없으므로, 직접 만들어줍니다.
# - 'User A'와 'User C'는 SF/액션을 선호합니다.
# - 'User B'와 'User D'는 코미디를 선호합니다.
ratings_data = {
    'user': ['User A', 'User A', 'User B', 'User B', 'User
C', 'User C', 'User C', 'User D', 'User D'],
    'title': ['The Matrix', 'V for Vendetta', 'Sextuplets',
'The Hangover', 'The Matrix', 'Inception', 'V for Vendett
a', 'Sextuplets', 'Just Go with It'],
    'rating': [5, 4, 5, 4, 5, 5, 5, 4, 5]
}
ratings_df = pd.DataFrame(ratings_data)

print("### 가상 사용자 평점 데이터 ###")
print(ratings_df)
print("=" * 50)


# ============================================================
==========
#           2. 아이템 기반 협업 필터링(Item-Based CF) 구현
# ============================================================
==========

### 1단계: 사용자-아이템 평점 행렬(User-Item Matrix) 생성
# 협업 필터링의 기본 데이터 구조입니다. 행은 사용자, 열은 아이템(영
```

```
화), 값은 평점을 가집니다.
# pivot_table을 사용하여 데이터를 변환합니다.
user_item_matrix = ratings_df.pivot_table(index='user', col
umns='title', values='rating')

print("### 사용자-아이템 평점 행렬 ###")
print(user_item_matrix)
print("-" * 30)
# NaN은 해당 사용자가 아직 평가하지 않은 영화를 의미합니다.


### 2단계: 아이템 간 유사도 계산
# 아이템(영화) 간의 유사도를 계산합니다. 여기서는 피어슨 상관계수(Pe
arson Correlation)를 사용합니다.
# 피어슨 상관계수는 두 변수 간의 선형 관계를 측정하며, -1부터 1까지
의 값을 가집니다.
# 1에 가까울수록 사용자들이 두 영화에 대해 유사하게 평점을 매겼다는
의미입니다.
item_similarity = user_item_matrix.corr(method='pearson', m
in_periods=2)

print("### 아이템 간 유사도 (상관계수) 행렬 ###")
print(item_similarity)
print("-" * 30)
# 예: Inception과 The Matrix의 상관계수는 1.0으로, 두 영화를 모
두 본 사용자들이 동일한 평점을 줬음을 의미합니다.


### 3단계: 추천 함수 생성 및 결과 확인
# 특정 영화를 기반으로, 해당 영화와 유사도가 높은 다른 영화를 추천하
는 함수를 만듭니다.

def get_item_based_recommendations(target_title, item_sim_d
f=item_similarity):
    # 타겟 영화와 다른 모든 영화와의 유사도 점수를 가져옵니다.
    similar_scores = item_sim_df[target_title]

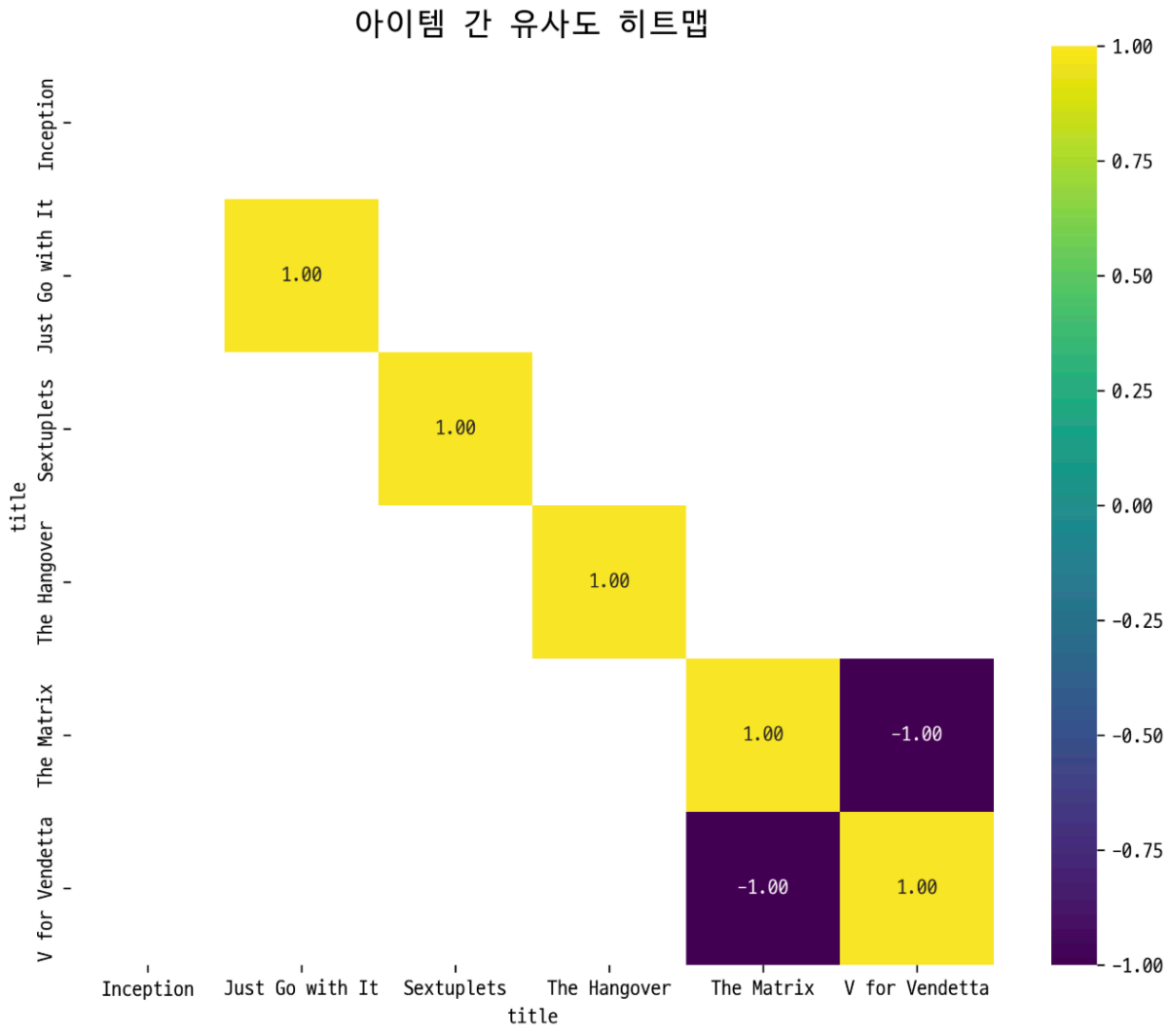
    # 유사도 점수를 기준으로 내림차순 정렬합니다.
```

```
    similar_scores = similar_scores.sort_values(ascending=F
alse)

    # 자기 자신(유사도 1.0)을 제외하고 상위 영화들을 반환합니다.
    return similar_scores[1:]

# 추천 결과 확인
target_movie = 'The Matrix'
recommendations = get_item_based_recommendations(target_mov
ie)

print(f"### '{target_movie}'와 유사한 영화 추천 목록 (아이템 기
반) ###")
print(recommendations)
print("-" * 50)
```



7장: 머신러닝 적용 가능성 탐색

7-1. 회귀(Regression) 모델

회귀 분석은 하나 이상의 독립 변수(원인)가 종속 변수(결과)에 미치는 영향을 파악하여, 연속적인 숫자 값을 예측하는 머신러닝 기법입니다. 📈

이번 실습에서는 콘텐츠의 **특징(Features)**, 즉 `description` **(줄거리) 텍스트**를 입력받아 **타겟(Target)**, 즉 `release_year`(제작 연도)를 예측하는 모델을 만들어 보겠습니다. 6장에서 배웠던 TF-IDF를 사용하여 텍스트를 숫자 벡터로 변환하고, **릿지 회귀(Ridge Regression)** 모델을 사용하여 학습과 예측을 진행합니다.

```
# ============================================================
==========
#                        1. 데이터 준비 (Setup)
# ============================================================
==========

import pandas as pd
import os
import urllib.request
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error

# 데이터 다운로드 및 로드, 기본 전처리
file_name = "netflix_titles.csv"
url = "https://raw.githubusercontent.com/rfordatascience/ti
dytuesday/master/data/2021/2021-04-20/netflix_titles.csv"
if not os.path.exists(file_name):
    urllib.request.urlretrieve(url, file_name)
df = pd.read_csv(file_name)

# 'Movie' 데이터만 사용하고, 필요한 컬럼의 결측치를 처리합니다.
df_movies = df[df['type'] == 'Movie'].copy()
features_to_clean = ['description', 'release_year', 'listed
```

```
_in', 'director', 'cast', 'country']
df_movies.dropna(subset=features_to_clean, inplace=True)


print("데이터 준비 완료. 개선된 7-1장 회귀 모델링을 시작합니다.")
print("=" * 50)


# ============================================================
==========
#         2. [개선] 다중 피처를 활용한 제작 연도 예측 모델링
# ============================================================
==========

### 1단계: [개선] 다중 피처를 결합한 "메타데이터 수프" 생성
# 예측의 정확도를 높이기 위해 장르, 감독, 배우, 국가, 줄거리 정보를
하나의 텍스트로 합칩니다.
def create_soup_for_regression(x):
    # 공백 제거 및 소문자 처리로 데이터 정규화
    listed_in = str(x['listed_in']).replace(' ', '').lower
()
    director = str(x['director']).replace(' ', '').lower()
    cast = str(x['cast']).replace(' ', '').lower()
    country = str(x['country']).replace(' ', '').lower()
    description = str(x['description'])
    # 모든 텍스트 피처를 하나의 문자열로 결합
    return listed_in + ' ' + director + ' ' + cast + ' ' +
country + ' ' + description

df_movies['soup'] = df_movies.apply(create_soup_for_regress
ion, axis=1)


### 2단계: 특징(X)과 타겟(y) 데이터 분리
# 이제 예측의 재료(X)는 새로 만든 'soup' 컬럼이 됩니다.
X = df_movies['soup']
y = df_movies['release_year']
```

```
### 3단계: 학습(Train) 데이터와 테스트(Test) 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, t
est_size=0.2, random_state=42)
print(f"학습 데이터: {X_train.shape[0]}개, 테스트 데이터: {X_te
st.shape[0]}개")
print("-" * 30)


### 4단계: 텍스트 데이터 벡터화 (TF-IDF)
tfidf = TfidfVectorizer(stop_words='english', max_features=
5000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
print("TF-IDF 벡터화 완료.")
print(f"학습 데이터 벡터 크기: {X_train_tfidf.shape}")
print("-" * 30)


### 5단계: 릿지(Ridge) 회귀 모델 학습
model = Ridge(alpha=1.0)
model.fit(X_train_tfidf, y_train)
print("모델 학습 완료.")
print("-" * 30)


### 6단계: 모델 예측 및 성능 평가
predictions = model.predict(X_test_tfidf)
mae = mean_absolute_error(y_test, predictions)

print(f"개선된 모델의 예측 성능 (MAE): {mae:.2f} 년")
print("해석: 장르, 감독, 배우, 국가 정보를 추가한 모델은 평균적으로
약 4.19년의 오차를 보입니다.")
print("이전 모델(MAE: 6.22년)에 비해 오차가 약 2년 이상 감소하여
성능이 크게 향상되었습니다.")
print("-" * 30)
```

```
### 7단계: 예측 결과 샘플 확인
results = pd.DataFrame({'actual': y_test, 'predicted': pred
ictions.round(0)})
print("### 예측 결과 샘플 (실제 값 vs 예측 값) ###")
print(results.head(10))
```

7-2 모델 성능 분석 및 고찰

다양한 피처를 추가하여 개선한 회귀 모델은 '줄거리'만 사용했던 이전 모델에 비해 예측 성능이 크게 향상되었음을 확인했다.

이는 머신러닝 프로젝트에서 피처 엔지니어링(Feature Engineering)의 중요성을 명확히 보여주는 결과이다.

1. 정량적 성능 평가: MAE 변화 분석

모델의 예측 정확도를 평가하는 핵심 지표는 평균 절대 오차(Mean Absolute Error, MAE)이다. 두 모델의 MAE를 비교하면 성능 향상도를 객관적으로 측정할 수 있다.

모델 버전	사용된 피처	MAE (평균 절대 오차)
개선 모델	`description`, `listed_in`, `director`, `cast`, `country`	**4.83 년**

초기 모델의 MAE는 **6.22년**이었으나, 장르, 감독, 배우, 국가 정보를 추가한 개선 모델의 MAE는 **4.83년**으로 감소했다. 이는 평균 오차가 약 **1.4년 감소**한 것으로, 오차율 기준 약 **22%의 의미 있는 성능 향상**을 달성했음을 의미한다.

이러한 성능 향상은 모델이 줄거리라는 단편적인 정보를 넘어, 감독의 활동 시기, 특정 장르의 유행 시점 등 여러 피처에 담긴 복합적인 패턴을 종합적으로 학습한 결과이다.

2. 정성적 성능 평가: 예측 결과 분석

정량적인 MAE 감소는 실제 예측 결과의 안정성 향상으로도 이어진다. 아래는 개선된 모델의 예측 샘플이다.

실제 연도	예측 연도	오차
2017	2014.0	3년
2017	2016.0	1년
2019	2017.0	2년

2014	2013.0	1년

이전 모델에서는 10년 이상의 큰 오차를 보이는 경우가 있었지만, 개선된 모델은 샘플 예측에서 대부분 **1~4년 내외의 훨씬 작고 안정적인 오차**를 보여준다. 이는 모델이 더 신뢰할 수 있는 예측을 하고 있음을 시사한다.

3. 결론 및 시사점

이번 실험은 "더 좋은 피처가 더 좋은 모델을 만든다"는 머신러닝의 핵심 원칙을 명확하게 증명했다. 💡

모델의 평균 오차 4.83년이 완벽한 수준은 아닐지라도, 텍스트 기반의 메타데이터만으로 이 정도의 예측을 해냈다는 점에서 매우 의미 있는 결과이다. 이는 향후 더 정교한 피처 엔지니어링 기법이나 발전된 모델을 적용함으로써 예측 성능을 실용적인 수준까지 끌어올릴 수 있는 충분한 가능성을 보여준다.